Natural Language Processing (NLP)

Haythem Ghazouani



Sommaire

1. Préambule
2. Pipeline
3. Modèles
4. Etude de cas

Préambule

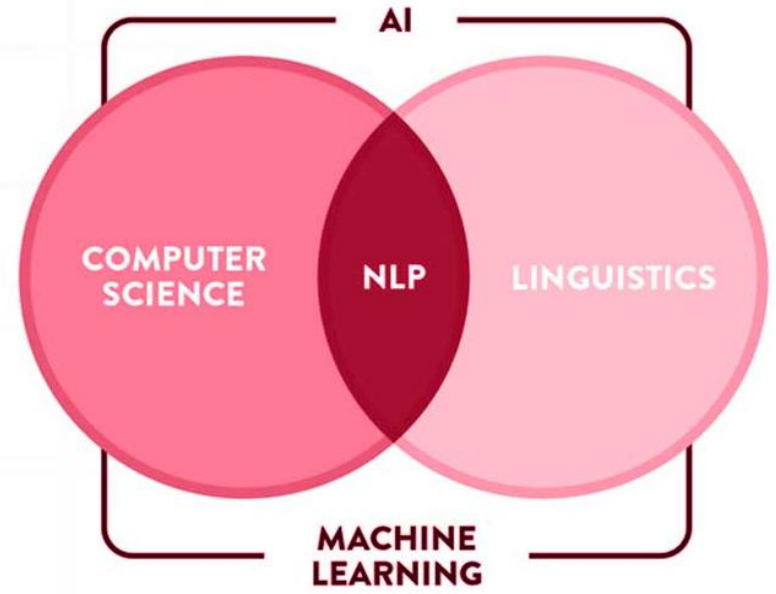


Préambule

1. Quésaco

NLP : Natural Language Processing = Traitement des Langages Naturels

- Par **langage naturel**, nous faisons référence au langage qui est utilisé par les êtres humains pour la communication. Il peut être du texte ou de la parole.
- Le **NLP** est l'intersection de trois champs :
  - ✓la science de l'informatique
  - ✓la linguistique
  - ✓l'IA
- Il s'agit essentiellement d'apprendre aux ordinateurs à comprendre, interpréter et générer le langage humain. Ceci correspond à réaliser un cycle automatisé de traitement et d'analyse des données sémiologiques de différentes structures phonologiques et/ou morphologiques.



Préambule

2. Contexte

Données textuelles en ligne

- 70 milliards web-pages online (1.9 billion sites web)
- 55 millions articles Wikipédia

Croissance à un rythme rapide

- 9000 tweets/seconde
- 3 millions emails/seconde (60% spam)

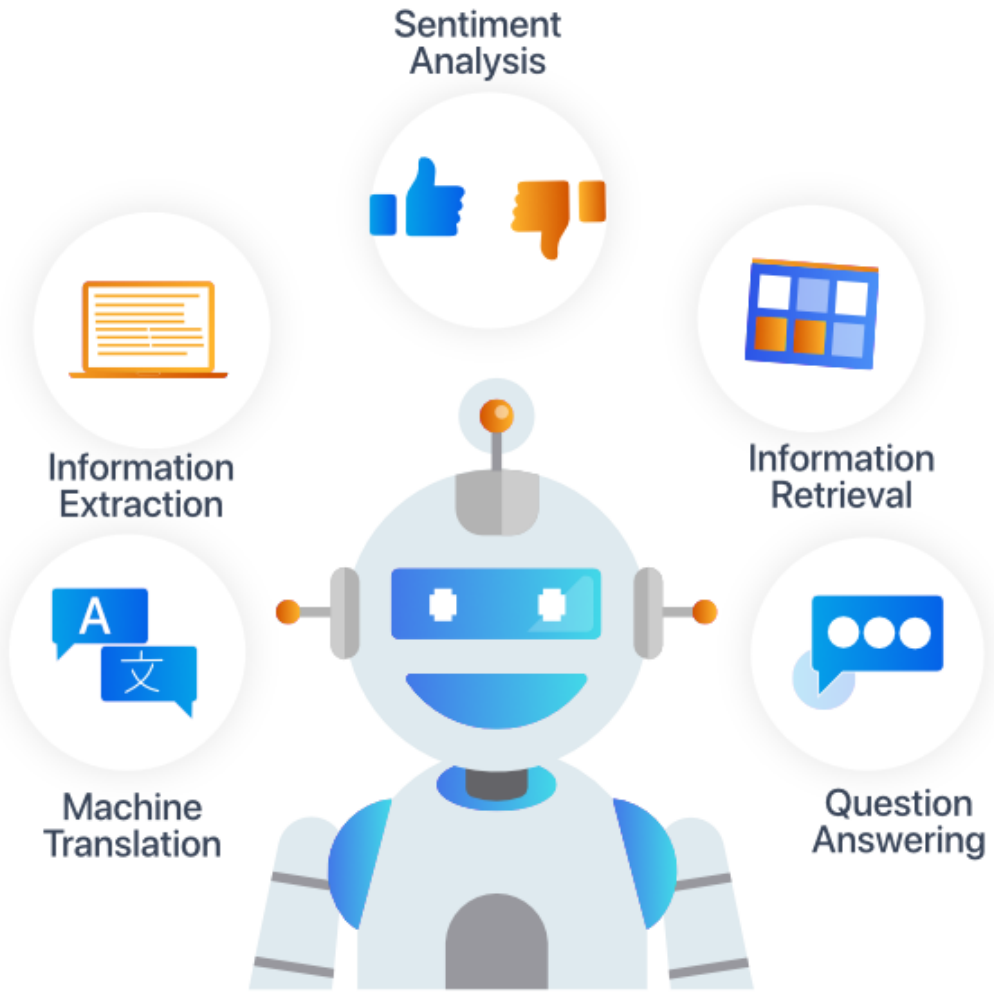
Utilisateurs potentiels

- 7,9 milliards de personnes utilisent une sorte de langue (janvier 2022)
- 4,7 milliards d'internautes (janvier 2021) (~59%)
- 4,2 milliards d'utilisateurs de réseaux sociaux (janvier 2021) (~54 %)

Préambule

3. Applications

- Moteur de recherche
- Récolte des avis sur le web
- Traduction
- Analyse d'opinions/sentiment
- Chatbots
- Filtre des emails
- Reconnaissance vocale
- Extraction de l'information
- Résumé d'un contenu
- …



Préambule

4. Historique

- Le NLP a traversé principalement 3 grandes époques :
  - ✓1940-1980 : Méthodes linguistiques et règles manuscrites (Méthodes symboliques)
  - ✓1980-2010 : Méthodes statistiques (SVM, RF, …)
  - ✓2010-aujourd'hui : Apprentissage en profondeur

- **1949 – Premier "modèle" de traduction automatique** (théorie de l'information de Shannon pour définir la traduction automatique comme un problème cryptographique)
- **1964 – ELIZA, premier bot conversationnel**
- **2012 : DL fait des percées majeures en NLP**

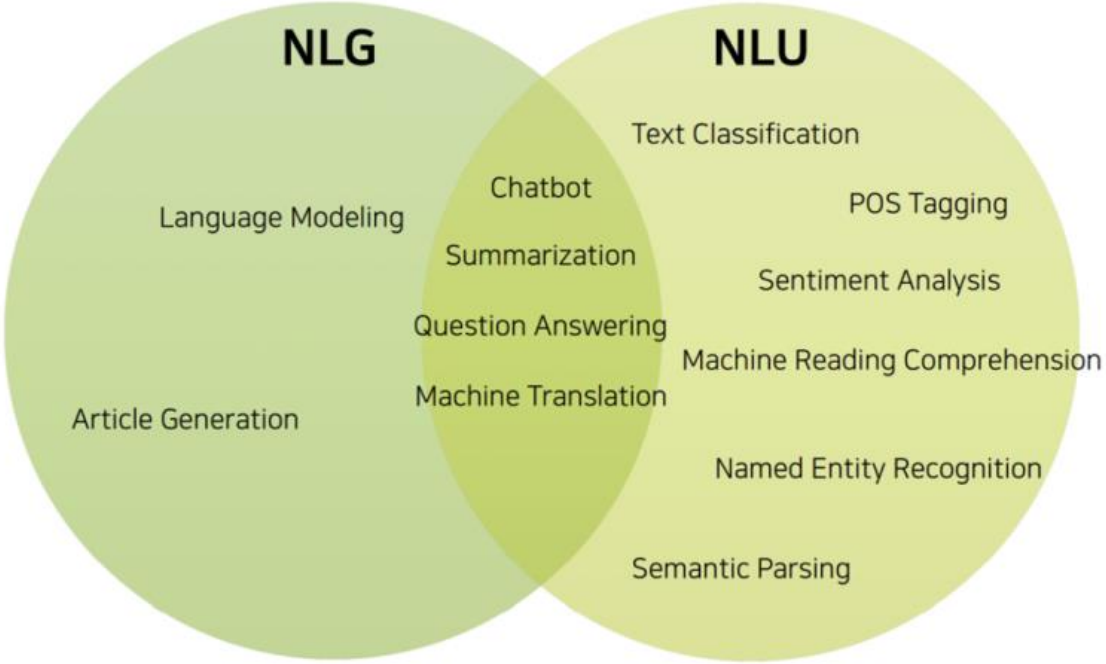


Préambule

5. Composants

- Deux composants principaux :
  - ✓Natural Language Understanding (NLU) : compréhension du langage naturel
  - ✓Natural Language Generation (NLG) : génération du langage naturel (qui nécessite NLU)

Mappage
de l'espace de représentation
informatique
à l'espace du langage

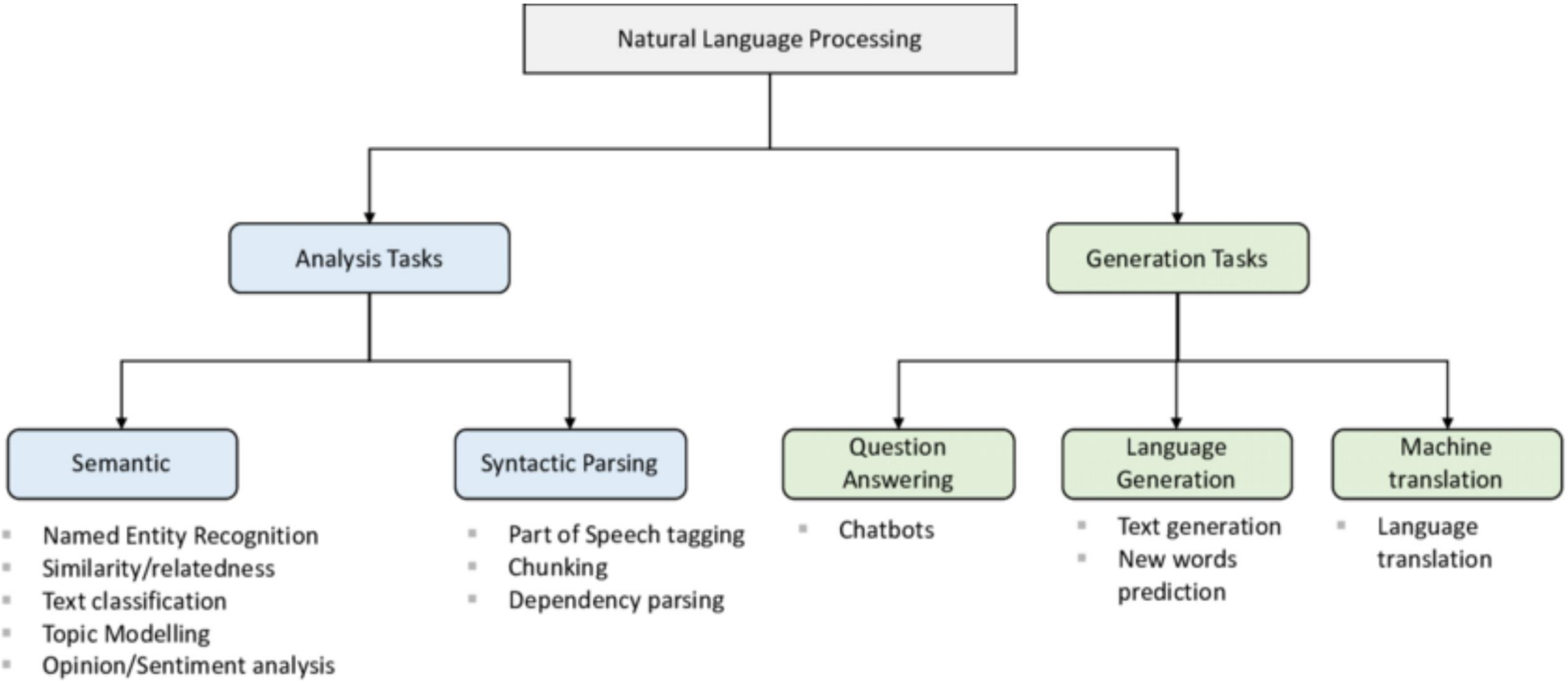


Mappage
de l'espace du langage
à l'espace de représentation
informatique (Concepts)

Préambule

5. Composants

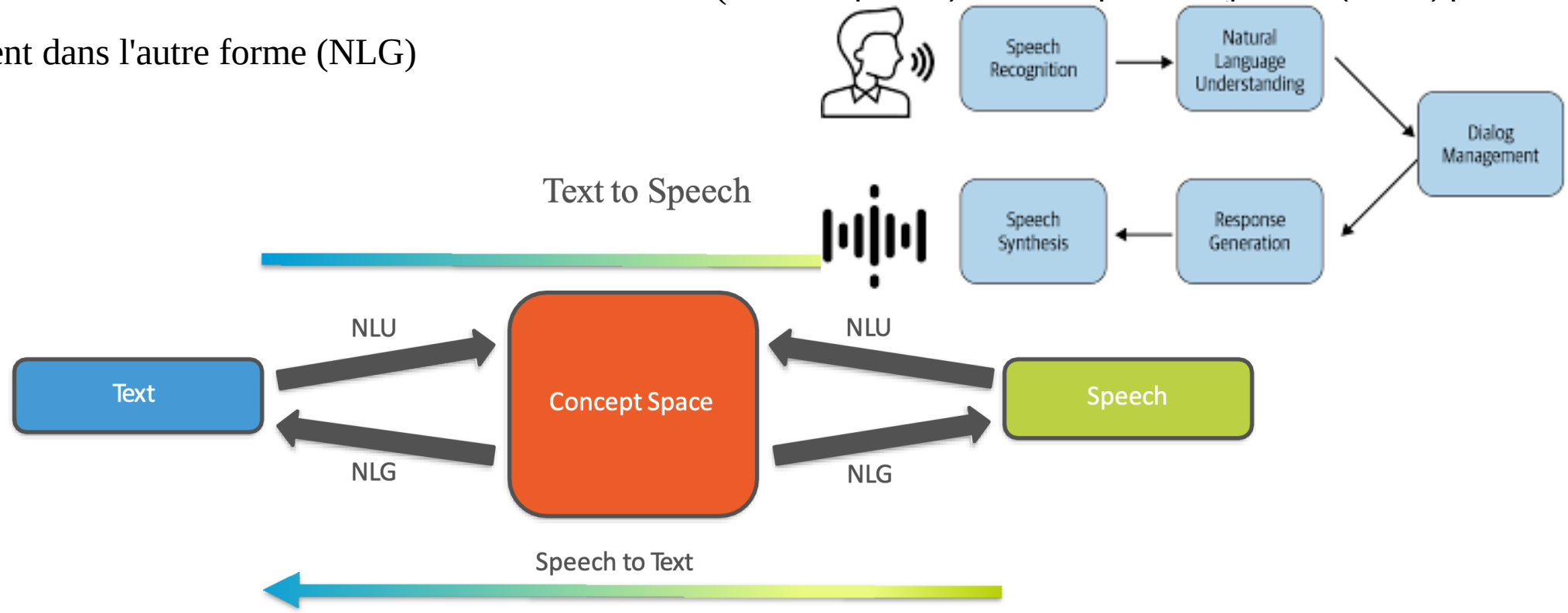
- NLP = NLU+NLG



Préambule

5. Composants

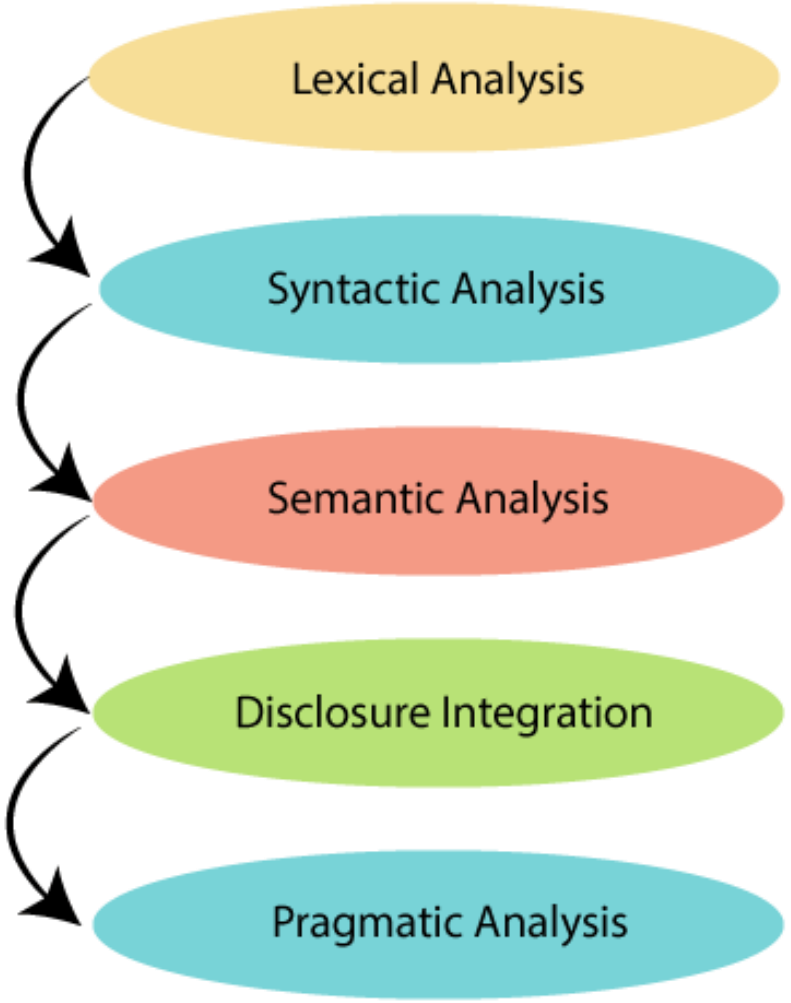
- Le langage naturel peut être du texte ou de la parole.
- L'objectif des deux est le même : traduire des données brutes (texte ou parole) en concepts sous-jacents (NLU) puis éventuellement dans l'autre forme (NLG)



Préambule

6. Niveaux

- Le NLP a 5 niveaux :



Préambule

6. Niveaux

- Le NLP a 5 niveaux :
  - ✓ **Analyse lexicale** : Identifier et analyser la structure des mots. Nous divisons tout le morceau de texte en paragraphes, phrases et mots.

    Elle se fait en se référant a la langue analysée, pour vérifier les mots

    ☑ It is

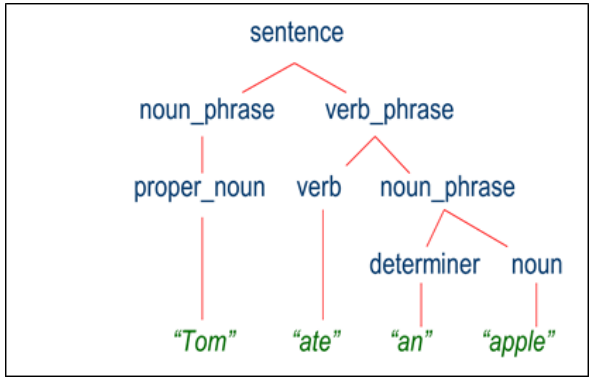
    ✗ Itis

Préambule

6. Niveaux

- Le NLP a 5 niveaux :
  - ✓ **Analyse lexicale** : Identifier et analyser la structure des mots. Nous divisons tout le morceau de texte en paragraphes, phrases et mots.
  - ✓ **Analyse syntaxique** : Analyser les mots dans les phrases pour la grammaire et les réorganiser pour déterminer comment ils sont liés les uns aux autres

Elle se base sur « Parse tree » pour déterminer la nature des mots



Préambule

6. Niveaux

- Le NLP a 5 niveaux :
  - ✓ **Analyse lexicale** : Identifier et analyser la structure des mots. Nous divisons tout le morceau de texte en paragraphes, phrases et mots.
  - ✓ **Analyse syntaxique** : Analyser les mots dans les phrases pour la grammaire et les réorganiser pour déterminer comment ils sont liés les uns aux autres
  - ✓ **Analyse sémantique** : Extraire les significations du dictionnaire du texte pour vérifier la signification.

    Elle se base sur « Semantic Graph »

    ☑ She drinks some milk

    ✗. She drinks some books

Préambule

6. Niveaux

- Le NLP a 5 niveaux :
  - ✓ **Analyse lexicale** : Identifier et analyser la structure des mots. Nous divisons tout le morceau de texte en paragraphes, phrases et mots.
  - ✓ **Analyse syntaxique** : Analyser les mots dans les phrases pour la grammaire et les réorganiser pour déterminer comment ils sont liés les uns aux autres
  - ✓ **Analyse sémantique** : Extraire les significations du dictionnaire du texte pour vérifier la signification.
  - ✓ **Intégration du discours** : On analyse la phrase précédente pour deviner le sens de la phrase actuelle et de celle qui la suit.

    Elle vise a déterminer l'intention (intent recognition)

    Monkeys eat banana when **they** are wake up; they = monkey 🐵

    Monkeys eat banana when **they** are ripe; they = banana 🍌

Préambule

6. Niveaux

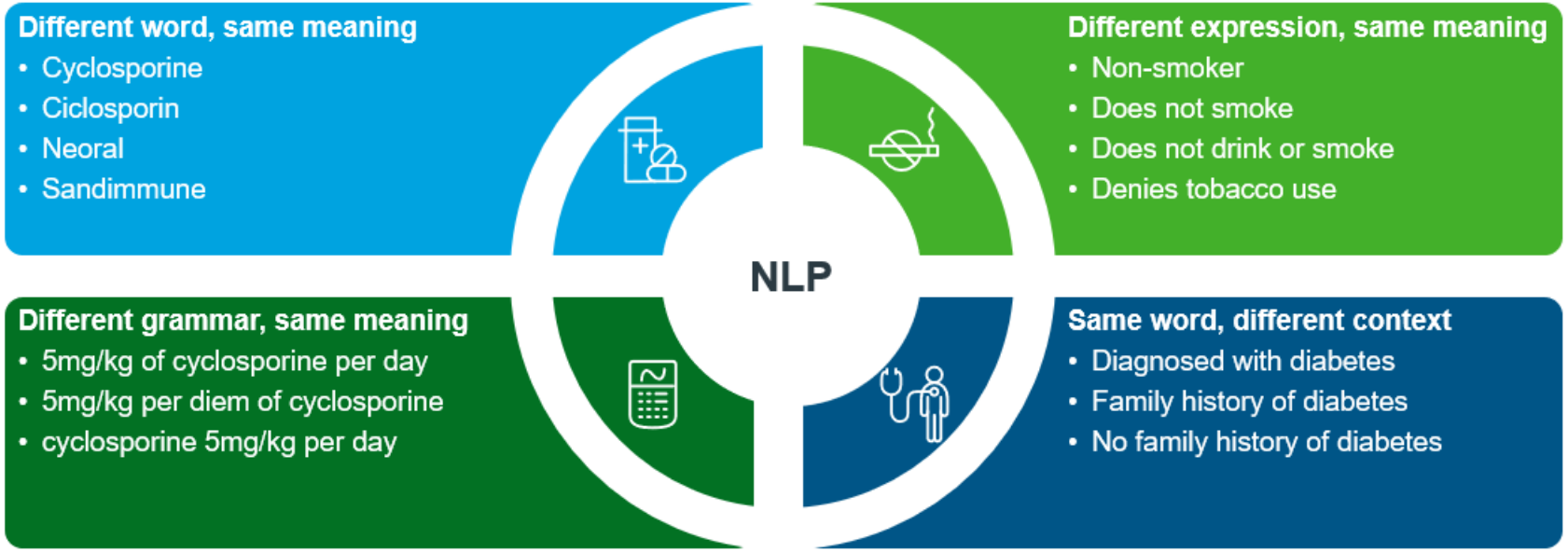
- Le NLP a 5 niveaux :
  - ✓ **Analyse lexicale** : Identifier et analyser la structure des mots. Nous divisons tout le morceau de texte en paragraphes, phrases et mots.
  - ✓ **Analyse syntaxique** : Analyser les mots dans les phrases pour la grammaire et les réorganiser pour déterminer comment ils sont liés les uns aux autres
  - ✓ **Analyse sémantique** : Extraire les significations du dictionnaire du texte pour vérifier la signification.
  - ✓ **Intégration du discours** : On analyse la phrase précédente pour deviner le sens de la phrase actuelle et de celle qui la suit.
  - ✓ **Analyse pragmatique** : Ceci essaie de récupérer les aspects de la langue qui nécessitent une connaissance du monde réel.

    Close the door → Order
    Please close the door → Request

Préambule

7. Défis





Préambule

7. Défis

Variabilité

Plusieurs langues existent et chacune d'elle a un ensemble de vocabulaires

Ambiguïté

les mots sont uniques mais peuvent avoir des significations différentes selon le contexte

Synonymie

nous pouvons exprimer la même idée avec différents termes qui dépendent également du contexte spécifique.

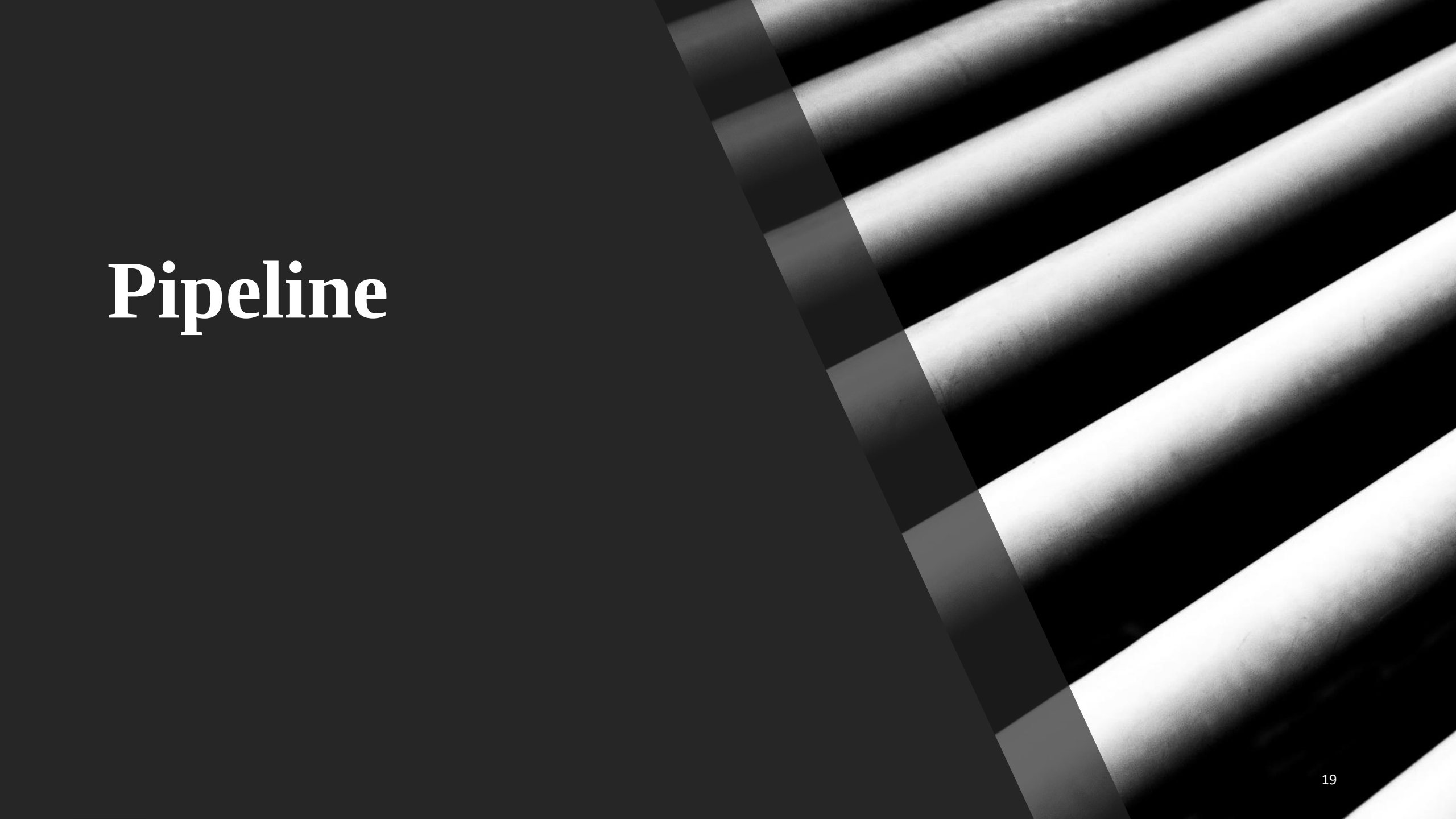
Coréférence

Les tâches de coréférence consistent à trouver toutes les expressions qui se réfèrent à la même entité.

Style d'écriture

Selon la personnalité de l'auteur, ses intentions et émotions, une même idée peut être exprimée de diverses manières. Certains auteurs n'hésitent pas à utiliser de l'ironie ou le sarcasme et donc véhiculer un sens opposé au sens littéral.

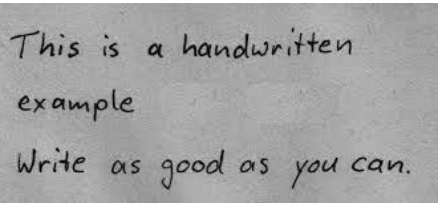
Pipeline



Système NLP









Audio → Speech Processing

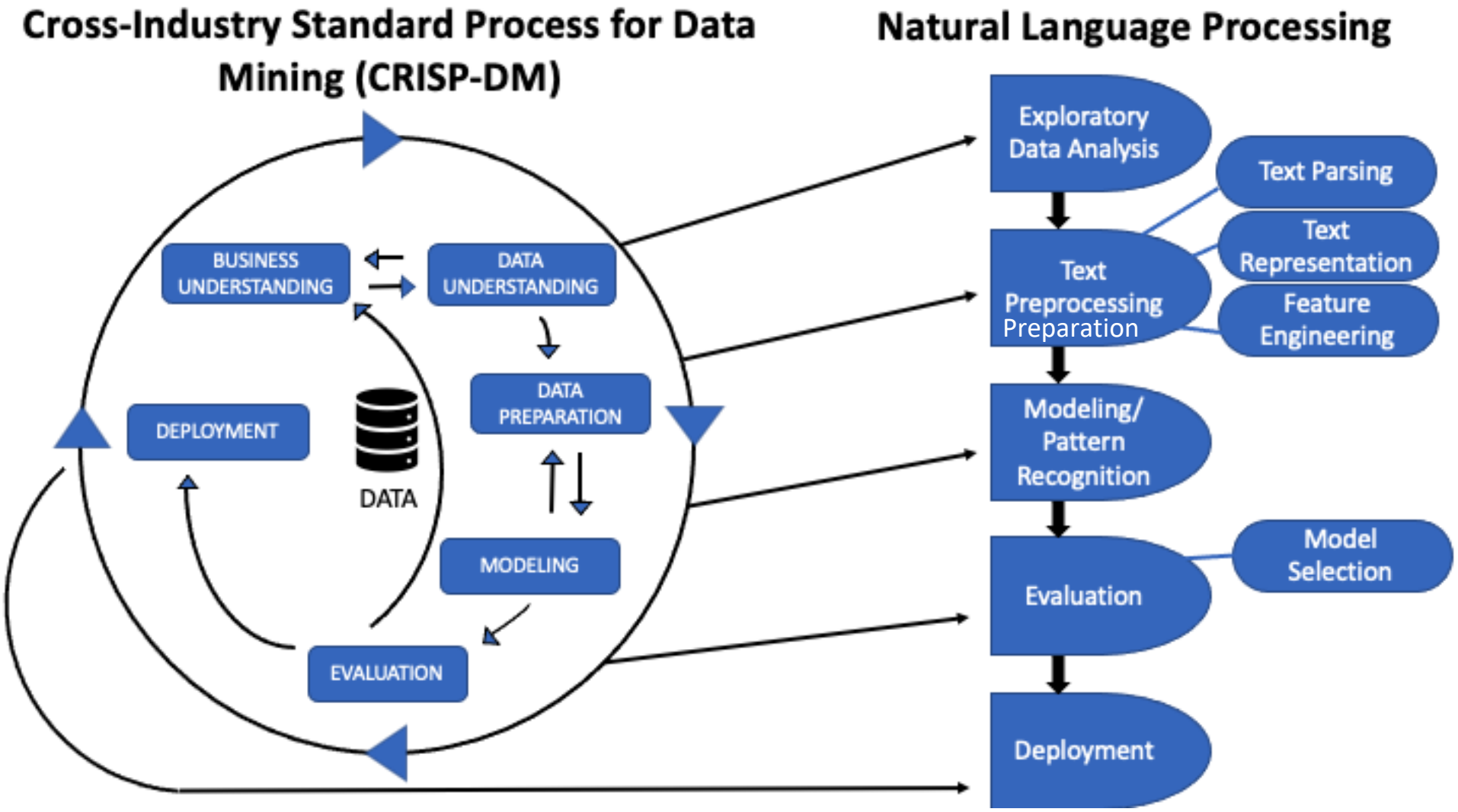
Image → OCR Processing

Text

- Document
- Sentence
- Word

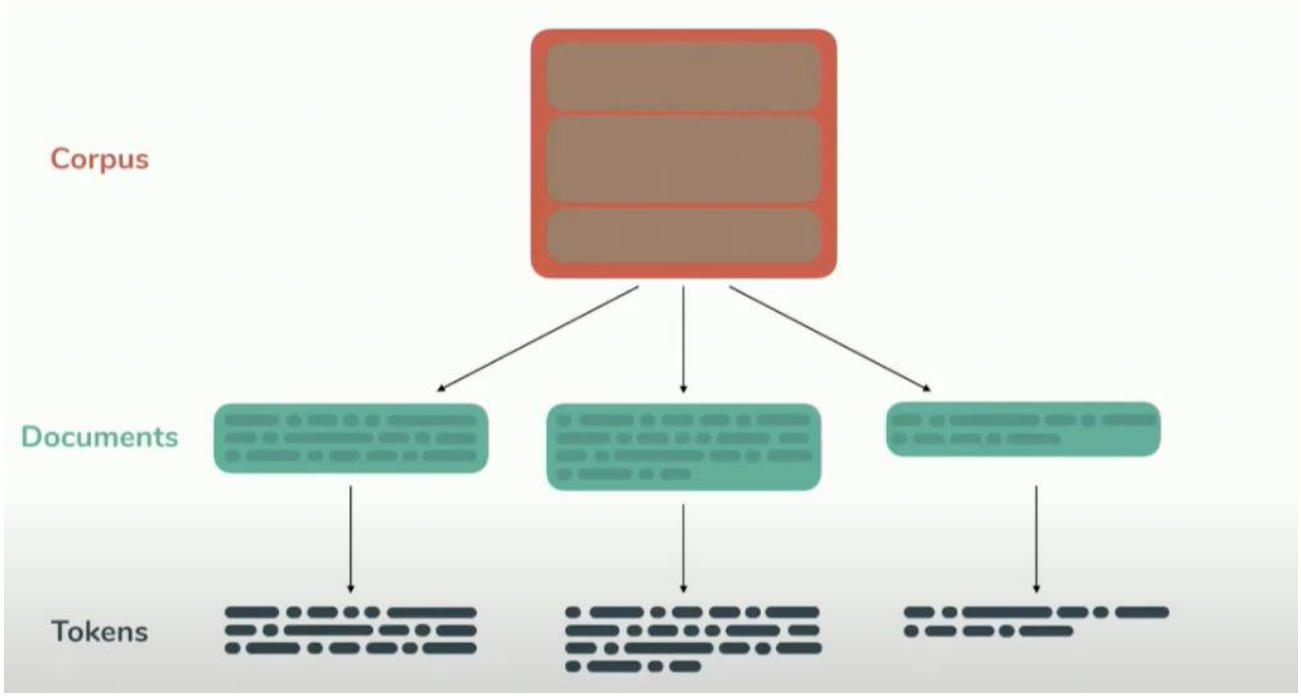
→ **NLP System** → **Output**

Système NLP


Cross-Industry Standard Process for Data Mining (CRISP-DM)
BUSINESS UNDERSTANDING
DATA UNDERSTANDING
DATA PREPARATION
MODELING
EVALUATION
DEPLOYMENT
DATA
Natural Language Processing
Exploratory Data Analysis
Text Preprocessing
Preparation
Text Parsing
Text Representation
Feature Engineering
Modeling/ Pattern Recognition
Evaluation
Model Selection
Deployment

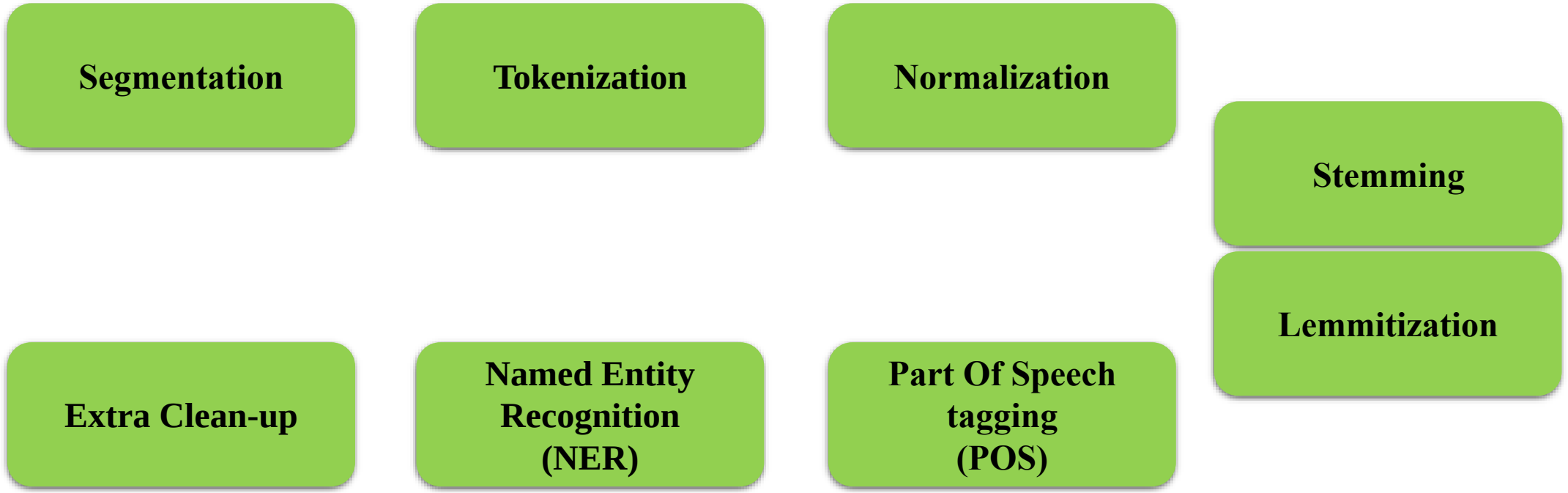
Terminologie

- **Corpus** est l'ensemble de textes d'un jeu de données.
- **Document** est un texte du corpus.
- **Séquence** ou **token** : Chaque document peut être découpé en unités de texte, qu'on appelle des séquences ou tokens qui peuvent être des mots, ou des caractères (des "lettres"), voire des morceaux de mots (des subwords).
- **Vocabulaire** est l'ensemble des séquences distinctes présentes dans un corpus (toutes les séquences uniques du corpus)



Pipeline du NLP

2. Preprocessing



Pipeline du NLP

2. Preprocessing

a) Segmentation (sequencing)

Elle permet de décomposer le document entier en des phrases (sequences).

Ceci est fait en se basant sur la ponctuation voire un caractère spécifique.

Pipeline du NLP

2. Preprocessing

b) Tokenisation

Chaque phrase est par la suite décomposée en des mots ou token. Cette étape peut être réalisée en se basant sur :

- **L'espace** : problème s'il y a des mots collés ou faute de frappe, exp. Hi, Hi!.

  Sinon les ponctuations, les préfixes, les suffixes et les apostrophes vont rester

- **L'expression régulière (regex)** : se contenter des mots d'une langue sans utilisation des ponctuations
- **Les outils dédiés** :
  - NLTK
  - Keras
  - spaCy

Pipeline du NLP

2. Preprocessing

c) Normalisation

Normalisation des mots : mettre les mots en minuscules (changement morphologique des mots, en arabe cela peut concerner le تشكيل):

- Pour avoir moins de vocabulaires
- Réduire les mots qui ne figurent pas dans le vocabulaire
- Il faut l'éviter si la forme du mot a un sens

Supprimer les mots non essentiels :

suppressions des mots qui ne sont pas essentiels et qui n'ajoutent pas beaucoup de sens a la phrase

- Pour avoir moins de vocabulaires

Pipeline du NLP

2. Preprocessing

d) La radicalisation - Stemming

Il s'agit de supprimer les préfixes et/ou suffixes de chaque mots. C'est une forme tronquée du mot qui peut modifier son sens.

Elle vise l'obtention de '**Word Stem**' de chaque mot. Le **Word Stem** se trouve dans le dictionnaire.

L'objectif de cette étape est la réduction du nombre de vocabulaires.

Exemple

approved

disapprove → **approve (stem)**

approves

Pipeline du NLP

2. Preprocessing

e) Lemmatisation

La lemmatisation est généralement réalisée en utilisant une table de correspondance contenant des formes lemmiques de mots en fonction de leur partie du discours, ainsi que des règles.

Cette procédure vise l'obtention de la racine d'un mot. Elle permet de trouver ainsi l'infinitif d'un verbe conjugué et le singulier pour un nom pluriel. La nouvelle forme du mot qui se trouve dans le dictionnaire et à partir de laquelle le mot est développé est donnée par **Root Stem** de **WordNet**.

Exemple

Went

Going → go (lemma)

Gone

Cette étape vise réduire énormément le nombre de vocabulaires, mais perd une certaine partie du contexte

Pipeline du NLP

2. Preprocessing

f) Marquage des mots – Part Of Speech tagging (POS)

Elle cherche à associer pour chaque mot son type grammatical (verbe, nom, déterminant, …). C'est une sorte de **classification** des mots.

Ceci se base sur les règles grammaticale et la réalisation de l'arbre syntaxique (parse/synthax tree). Cependant, elle peut être réalisée en se basant sur un modèle de ML.

A la fin, on ajoute des balises aux documents. C'est le balisage des parties des discours.

Exemple :

Give me you **answer** → Noun

Answer this question → Verb

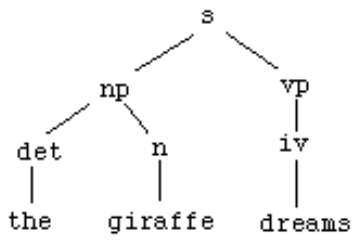
```
s    → np vp

np   → det n
vp   → tv np
     → iv

det  → the
     → a
     → an

n    → giraffe
     → apple

iv   → dreams
tv   → eats
     → dreams
```



Pipeline du NLP

2. Preprocessing

g) Reconnaissance des entités nommées – Named Entity Recognition (NER)

Elle vise la **classification** des mots en des sous-catégories :

- Personnes connues
- Quantités
- Lieu
- Organisations
- Noms des films
- Monnaie
- …

Elle peut être réalisée en se basant sur un modèle de ML.

Pipeline du NLP

2. Preprocessing

h) Extra nettoyage

Unifier les codes des caractères : Nous pouvons avoir des caractères non ascii et qui peuvent poser des problèmes.

Dans ce cas, il est nécessaire de décoder les caractères **non-Unicode** en une forme **normalisée** telle que UTF8

Suppression des caractères spéciaux : notamment pour les fichiers HTML issus du Web Scrapping

Pipeline du NLP

3. Préparation

a) Création du corpus

Cette étape doit être faite **obligatoirement** après la phase de preprocessing.

Exemple : Si notre corpus est constitué des documents suivants :

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

Alors, on obtient un vocabulaire de taille **N = 11 : {** 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'**}**

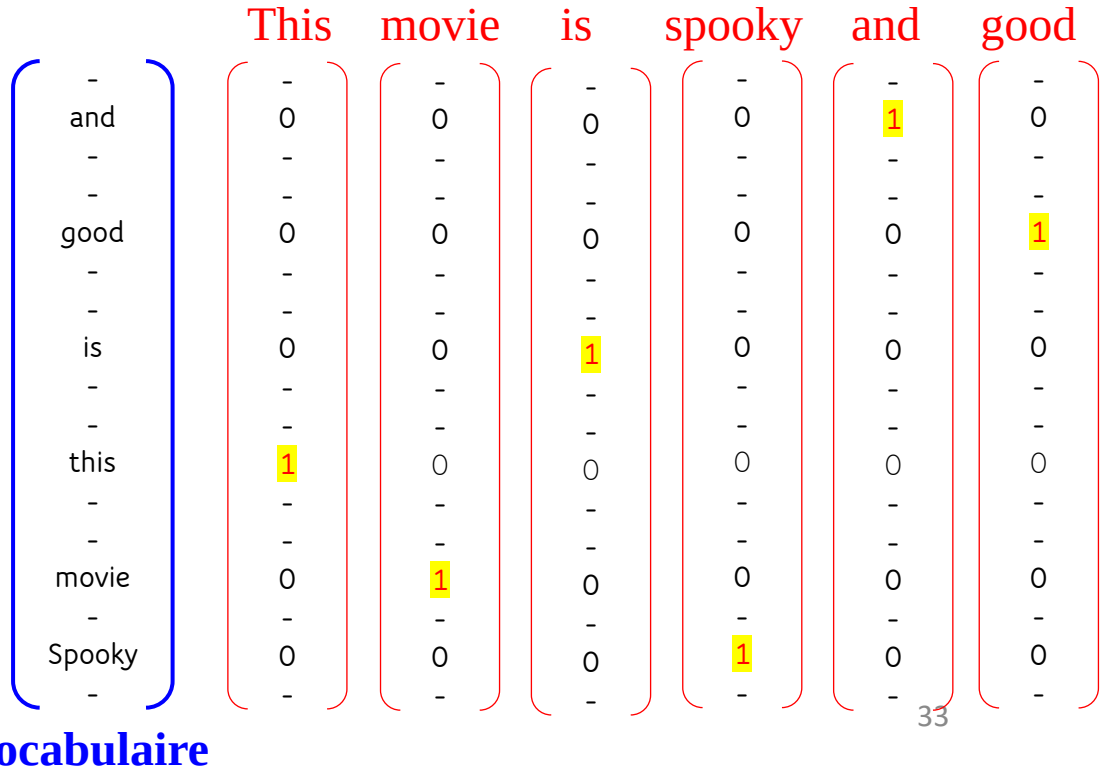
Pipeline du NLP

3. Préparation

b) Vectorisation

Comment représenter maintenant un document de manière numérique ?

Il est possible de le représenter par un ensemble de vecteurs one-hot encoder par rapport au vocabulaire (dictionnaire) du corpus.



Pipeline du NLP

3. Préparation

b) Vectorisation

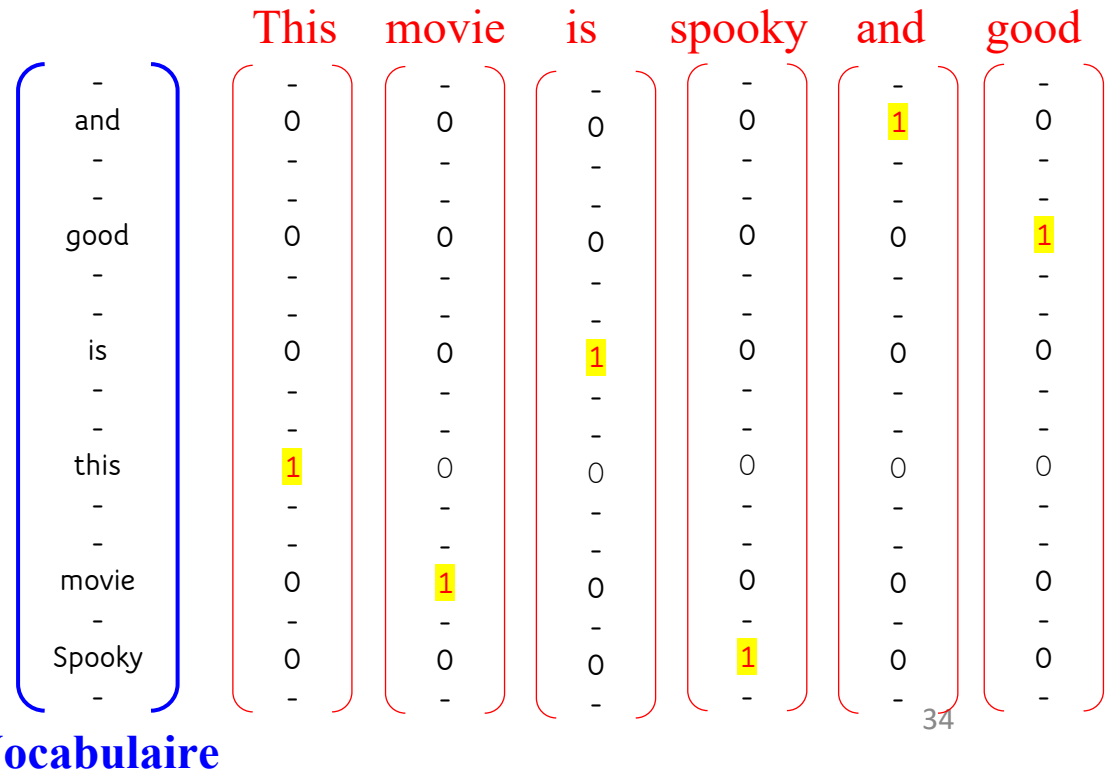
Comment représenter maintenant un document de manière numérique ?

Il est possible de le représenter par un ensemble de vecteurs one-hot encoder par rapport au vocabulaire (dictionnaire) du corpus.

La matrice qui représente un document est volumineuse et creuse

Il est possible de résoudre ce problème par le Vecteur de type **BoW**

(Bag of Words)



Pipeline du NLP

3. Préparation

b) Vectorisation

Bag-of-Words (BoW) est une approche qui permet de représenter un document par un vecteur. Elle permet aussi de créer le vocabulaire du corpus.

Principe :

Chaque document est transformé en un vecteur, dont les composantes sont fonction des occurrences de chaque séquence du vocabulaire dans le document en question. Ce vecteur est un tableau a une seule dimension.

Ceci peut être fait de plusieurs manières :

- Binarisation
- Occurrence
- Fréquence
- Fréquence inverse du document (TF-IDF)
- …

Pipeline du NLP

3. Préparation

b) Vectorisation : Binarisation

Le texte peut être encodé en se basant sur la binarisation. Les mots du vocabulaire sont considérés comme des features et on indique la présence ou non (1/0) de chaque mot dans le document correspondant

Exemple :

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

	1 This	2 movie	3 is	4 very	5 scary	6 and	7 long	8 not	9 slow	10 spooky	11 good	Length of the review(in words)
Review 1	1	1	1	1	1	1	1	0	0	0	0	7
Review 2	1	1	1	0	0	1	1	0	1	0	0	8
Review 3	1	1	1	0	0	0	1	0	0	1	1	6

Pipeline du NLP

3. Préparation

b) Vectorisation : Nombre d'occurrence (count)

Le texte peut être encodé en se basant sur le nombre d'apparition de chaque mot

Exemple :

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

	1 This	2 movie	3 is	4 very	5 scary	6 and	7 long	8 not	9 slow	10 spooky	11 good	Length of the review(in words)
Review 1	1	1	1	1	1	1	1	0	0	0	0	7
Review 2	1	1	2	0	0	1	1	0	1	0	0	8
Review 3	1	1	1	0	0	0	1	0	0	1	1	6

Pipeline du NLP

3. Préparation

b) Vectorisation : Fréquence

Le texte peut être encodé en se basant sur la fréquence de chaque mot (une sorte de normalisation)

Exemple :

- **Review 1: This movie is very scary and long**
- Vocabulaire ={ ‘This’, ‘movie’, ‘is’, ‘very’, ‘scary’, ‘and’, ‘long’, ‘not’, ‘slow’, ‘spooky’, ‘good’}
- Nombre de mots de Review 1 = 7
- Fréquence ‘This’ = (nombre de fois que « This » apparaît dans Review 1)/(nombre de termes de la Review 1) = 1/7
- Fréquence (‘movie’) = 1/7
- Fréquence (‘is’) = 1/7
- Fréquence (‘very’) = 1/7
- Fréquence (‘scary’) = 1/7
- Fréquence (‘and’) = 1/7
- Fréquence (‘long’) = 1/7
- Fréquence (‘not’) = 0/7 = 0
- Fréquence (‘slow’) = 0/7 = 0
- Fréquence (‘spooky’) = 0/7 = 0
- Fréquence (‘good’) = 0/7 = 0

Pipeline du NLP

3. Préparation

b) Vectorisation : Fréquence

Le texte peut être encodé en se basant sur la fréquence de chaque mot (une sorte de normalisation)

Exemple :

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

Term	TF (Review 1)	TF (Review 2)	TF (Review 3)
This	1/7	1/8	1/6
movie	1/7	1/8	1/6
is	1/7	1/4	1/6
very	1/7	0	0
scary	1/7	1/8	0
and	1/7	1/8	1/6
long	1/7	0	0
not	0	1/8	0
slow	0	1/8	0
spooky	0	0	1/6
good	0	0	1/6

Pipeline du NLP

3. Préparation

b) Vectorisation : TF-IDF (Term Frequency-Inverse Document Frequency)

Le texte peut être encodé en se basant sur la fréquence de chaque mot et de la nature du mot (mot fréquent ou non)

Le terme fréquent (TF) est une mesure de la fréquence à laquelle un terme, t, apparaît dans un document, d

$$tf_{t,d} = \frac{n_{t,d}}{Number\ of\ terms\ in\ the\ document}$$

Ceci dit que $n_{t,d}$ est le nombre de fois que le terme t apparaît dans le document d.

IDF est une mesure de l'importance d'un terme. Nous avons besoin de la valeur IDF car calculer uniquement le TF seul n'est pas suffisant pour comprendre l'importance des mots :

$$idf_t = \log \frac{number\ of\ documents}{number\ of\ documents\ with\ term\ 't'}$$

→ **Fréquence inverse du document**

$$TF\text{-}IDF_{t,d} = tf_{t,d} * idf_t$$

Pipeline du NLP

3. Préparation

b) Vectorisation : TF-IDF (Term Frequency-Inverse Document Frequency)

Le texte peut être encodé en se basant sur la fréquence de chaque mot et de la nature du mot (mot fréquent ou non)

Exemple :

Nous pouvons calculer les valeurs IDF pour tous les mots
IDF (‘This’) = log(nombre de documents/nombre de documents contenant le mot ‘this’) = log(3/3) = log(1) = 0
De la même manière :
IDF(‘movie’) = log(3/3) = 0
IDF(‘is’) = log(3/3) = 0
IDF(‘very’) = log(3/1) = log(3) = 0,48
IDF(‘scary’) = log(3/2) = 0,18
IDF(‘and’) = log(3/3) = 0
IDF(‘long’) = log(3/1) = log(3) = 0,48
IDF(‘not’) = log(3/1) = log(3) = 0,48
IDF(‘slow’) = log(3/1) = log(3) = 0,48
IDF(‘spooky’) = log(3/1) = log(3) = 0,48
IDF(‘good’) = log(3/1) = log(3) = 0,48

Term	IDF
This	0.00
movie	0.00
is	0.00
very	0.48
scary	0.18
and	0.00
long	0.48
not	0.48
slow	0.48
spooky	0.48
good	0.48

Pipeline du NLP

3. Préparation

b) Vectorisation : TF-IDF (Term Frequency-Inverse Document Frequency)

Le texte peut être encodé en se basant sur la fréquence de chaque mot et de la nature du mot (mot fréquent ou non)

Exemple :

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

Term	IDF	TF-IDF (Review 1)	TF-IDF (Review 2)	TF-IDF (Review 3)
This	0.00	0.000	0.000	0.000
movie	0.00	0.000	0.000	0.000
is	0.00	0.000	0.000	0.000
very	0.48	0.068	0.000	0.000
scary	0.18	0.025	0.022	0.000
and	0.00	0.000	0.000	0.000
long	0.48	0.068	0.000	0.000
not	0.48	0.000	0.060	0.000
slow	0.48	0.000	0.060	0.000
spooky	0.48	0.000	0.000	0.080
good	0.48	0.000	0.000	0.080

Pipeline du NLP

3. Préparation

b) Vectorisation : TF-IDF (Term Frequency-Inverse Document Frequency)

Le texte peut être encodé en se basant sur la fréquence de chaque mot et de la nature du mot (mot fréquent ou non)

Cette technique permet de :

- Réduire l'importance des termes présents partout
- Faire ressortir les termes discriminants

Pipeline du NLP

3. Préparation

b) Vectorisation : Evaluation

La technique de vectorisation utilisée Bag of Word (BoW) est :

+ Simple à implémenter

+ Rapide au niveau de l'exécution

+ Convenable pour un NN puisqu'elle assure la même taille pour tous les documents

+ Bien pour mesurer la similarité (la distance entre) des documents

- Incorrecte souvent pour mesurer la similarité entre les documents (exp. documents opposés via un seul token «non»)

- Indépendante du contexte, pas de notions de séquences ou d'ordre entre les mots

- Couteuse en termes d'attributs qui sont générés, le document est un vecteur immense mais creux (sparcity)

- Confrontée souvent au problème d'overfitting

Pipeline du NLP

3. Préparation

b) Vectorisation : Exemple

Lab Bow & pipeline

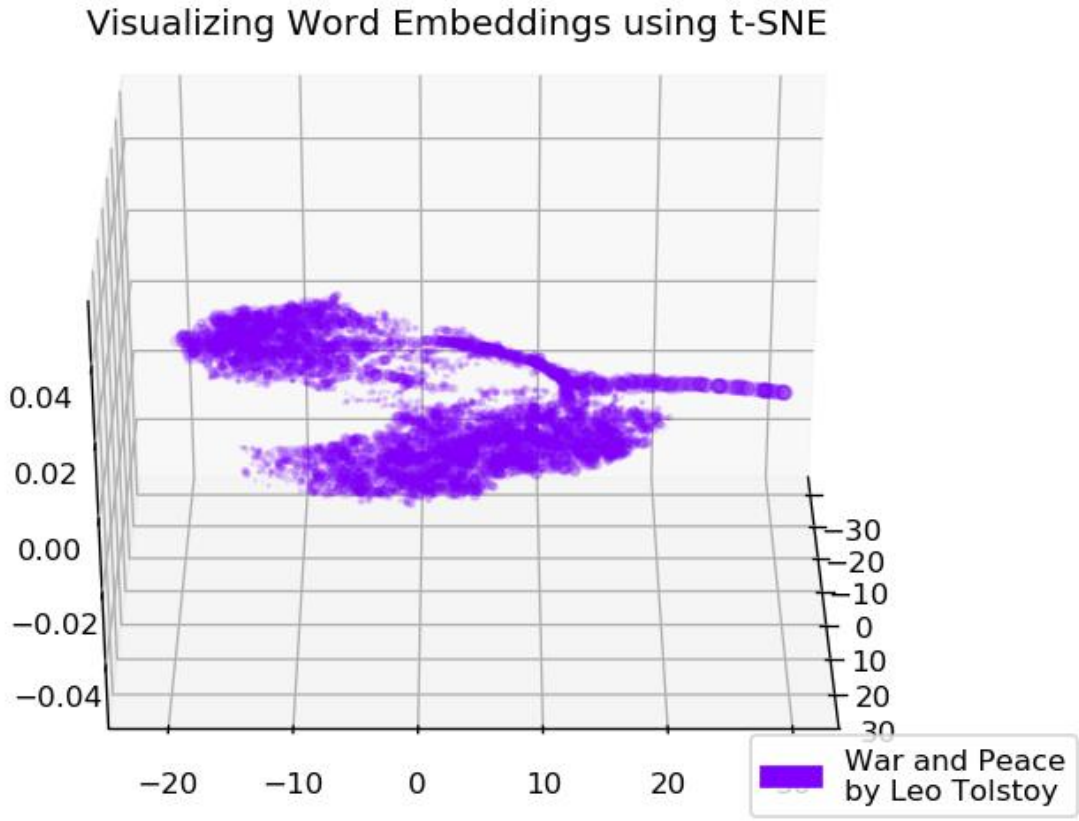
https://colab.research.google.com/drive/1Z4v8b-5nzrplDsXdDH9kzW7N_w5KtL0Y?usp=sharing

Deux parties existent :

A) Modèle basique

B) Modèle complet

Autre type de vectorisation



Word embeddings

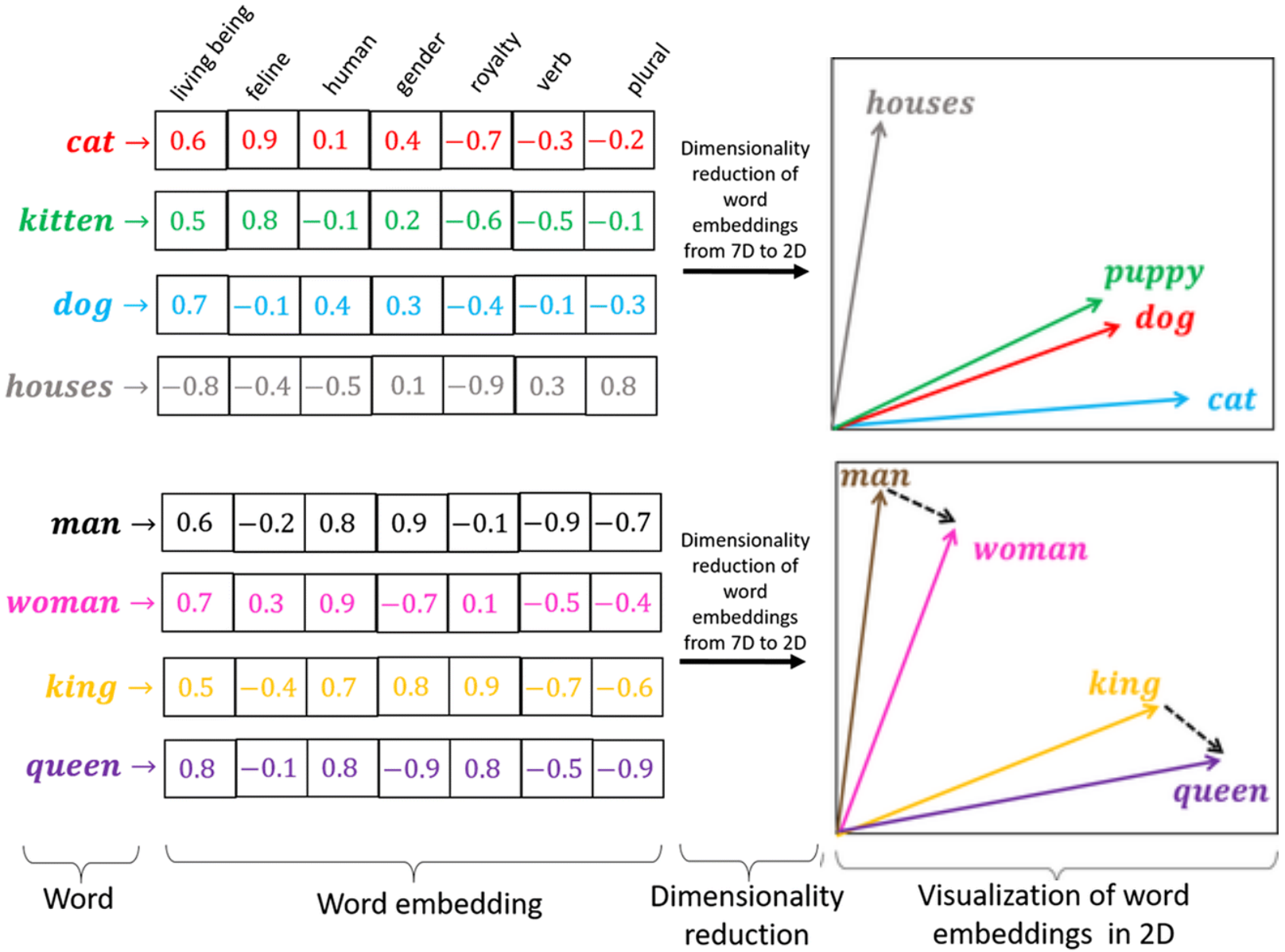
1. Principe

- Chaque **mot** (et non chaque phrase-BoW) sera représenté par un vecteur de IR^n qui est un vecteur dense qui donne les coordonnées du mot dans son contexte.
- Il s'agit de créer des caractérisations des mots dans un espace multidimensionnel, plus représentatif du sens des mots. En pratique, il faut rapprocher les mots similaires (comme les synonymes, ou ceux qui font partie du même champ lexical, …).
- Pour ce faire, il faut se baser sur un modèle d'apprentissage qui choisisse les caractéristiques les plus pertinentes représentant le mot. Par exemple, la caractéristique « être vivant » pourrait être intéressante pour différencier « chien » et « pc », chose qui permet de rapprocher « chien » et « chat » par la suite dans le traitement.

Word embeddings

1. Principe

Exemple



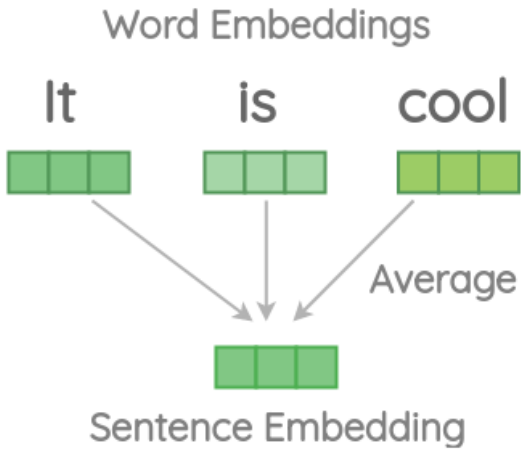
Word embeddings

1. Principe

- Chaque mot vecteur va avoir une centaine de dimensions au lieu de milliers.
- C'est une représentation vectorielle (mathématique) qui est en même temps cohérente avec la sémantique.
- Les propriétés algébriques sont applicables sur ce type de représentation vectorielle des mots. Elles donnent en plus un sens aux opérations algébriques effectuées.
- Cette technique est capable de réduire la dimension du mot, de capturer son contexte et de déterminer la similarité sémantique et syntaxique (genre, synonymes, …) du mot avec d'autres.
- La phrase sera alors représentée par l'ensemble de vecteurs (words embedding)

ou via la moyenne sur chaque composante ou via la somme,

ou même via le résultat du traitement de ces vecteurs par un RN (CNN, RNN, …)

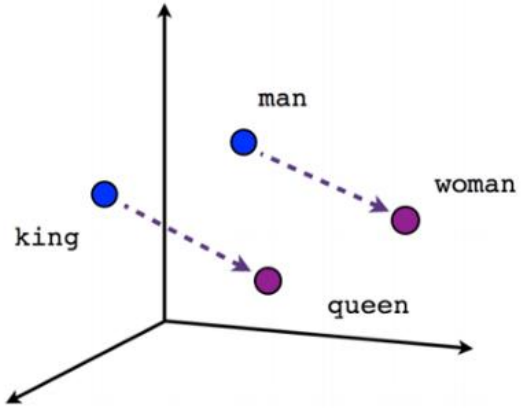


Word embeddings

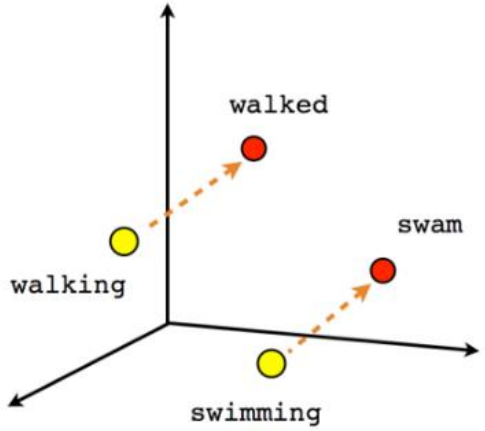
2. Aperçu

Que peut-on avoir alors comme Relations et Distances ?

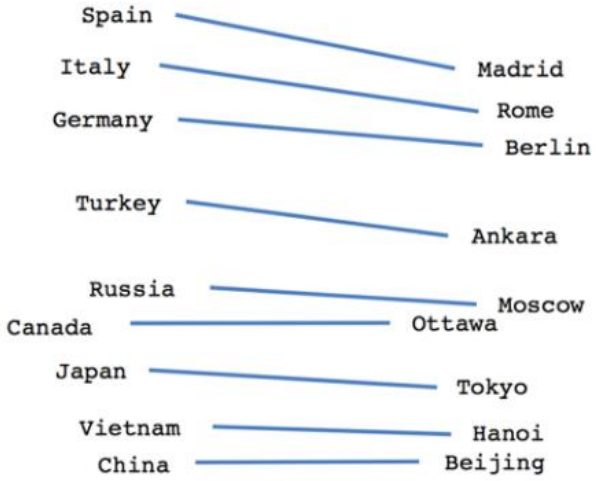
Voici un exemple :



Male-Female



Verb tense



Country-Capital

Si on effectue par exemple l'opération Spain – Madrid + Germany ≅ Berlin

Ceci dit que le modèle a pu capturé une partie du sens des mots pour les représenter dans un **espace**

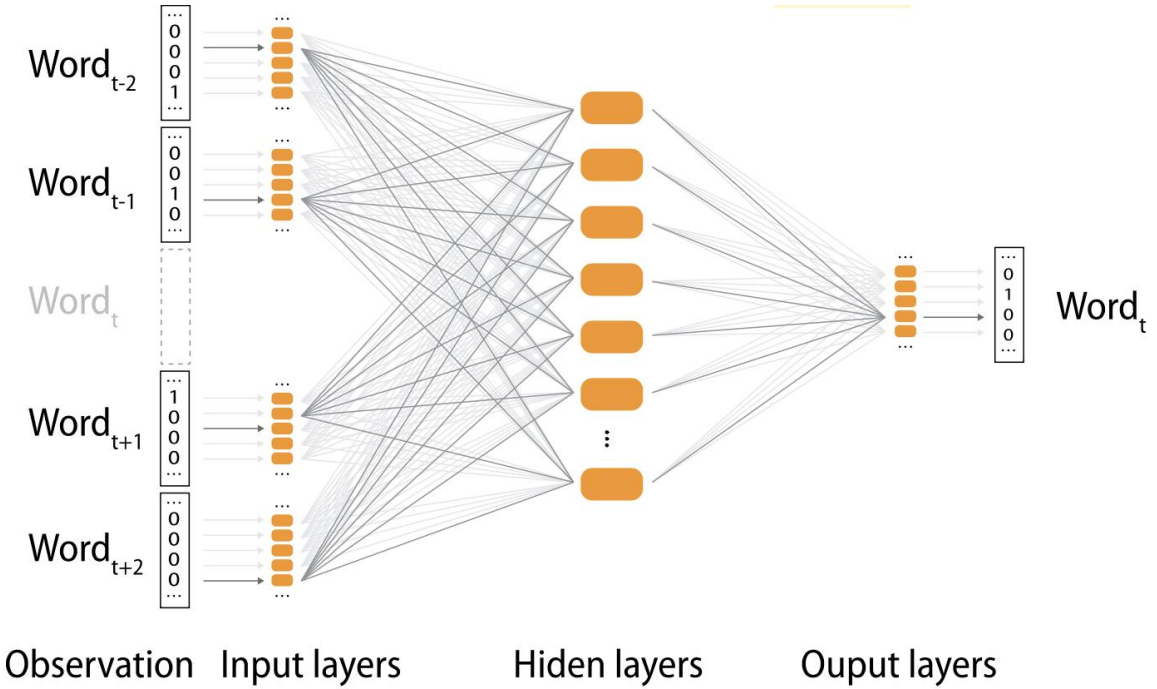
Word embeddings

2. Aperçu

Cet espace a été crée via un RN qui possède une seule couche cachée.

L'idée est d'utiliser les représentations vectorielles classiques du mot que l'on veut représenter et des mots l'entourant dans son contexte (la phrase). Il s'agit de compresser ainsi l'information sur la couche cachée du RN. Le nombre de neurone de la couche cachée représente la taille du mot compressé.

Exemple :

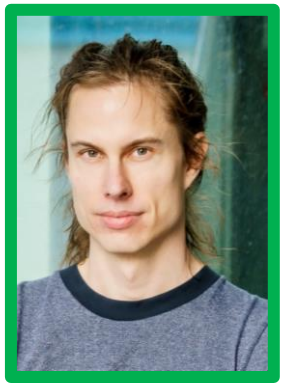


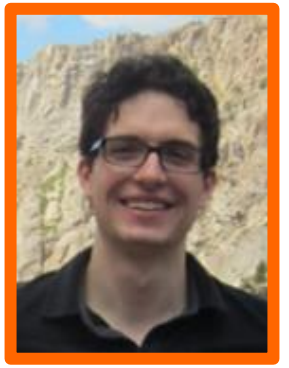
Word embeddings

2. Aperçu

Histoire

- Le concept n'est pas nouveau
- Années 60, VSM : Vector Space Model
- Années 80, LSA : Latent Semantic Analysis

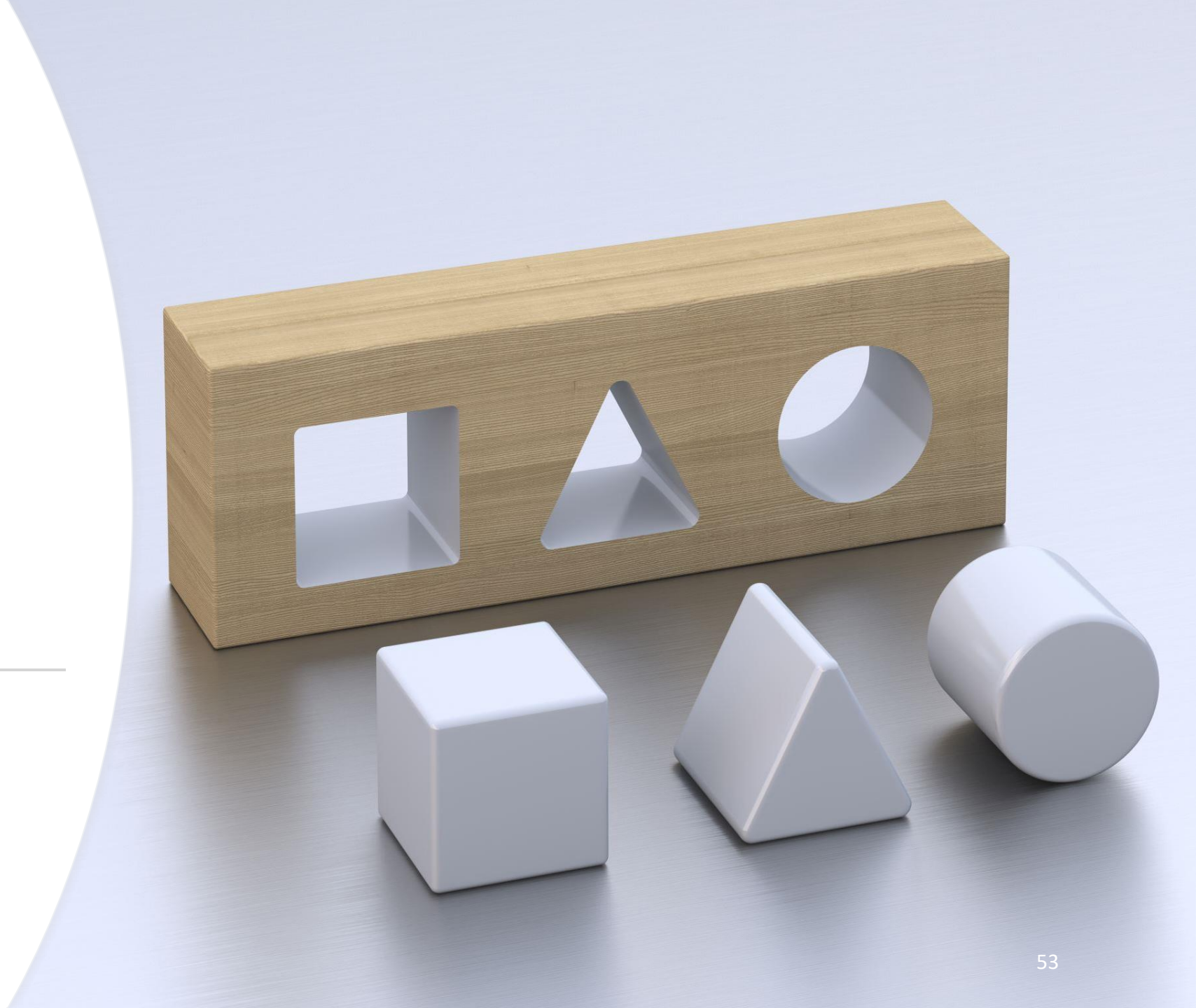




Mise en œuvre

- **Word2vec**, Thomas Mikolov (Google), 2013, NN
- **GloVe**, Jeffrey Pennington (Stanford), 2014, statistiques sur les cooccurrence des termes
- **FastText**, Thomas Mikolov AI Research (Facebbok), 2016
- **ELMo**, transformer
- …

Exemple de Keras



Exemple

1. **Tokenisation**

```
from tensorflow.keras.preprocessing.text import Tokenizer

sentences = [
    'i love my dog',
    'I, love my cat',
    'You love my dog!'
]

tokenizer = Tokenizer(num_words = 100)
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index
print(word_index)
```

```
{'love': 1, 'my': 2, 'i': 3, 'dog': 4, 'cat': 5, 'you': 6}
```

Exemple

1. Tokenisation

```
import tensorflow as tf
from tensorflow import keras


from tensorflow.keras.preprocessing.text import Tokenizer


sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100)
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)

print("\nWord Index = " , word_index)
print("\nSequences = " , sequences)
```

Corpus/documents

Vocabulaire

```
Word Index =  {'my': 1, 'love': 2, 'dog': 3, 'i': 4, 'you': 5, 'cat': 6, 'do': 7, 'think': 8, 'is': 9, 'amazing': 10}

Sequences =  [[4, 2, 1, 3], [4, 2, 1, 6], [5, 2, 1, 3], [7, 5, 8, 1, 3, 9, 10]]
```

La vectorisation du document au niveau de Keras :
le document est représenté par un vecteur qui contient des valeurs qui ne sont pas de type one-hot-encoder ou de type Bow.
Ces valeurs correspondent aux indices (ordre) des mots dans le vocabulaire.

Exemple

1. Tokenisation

```
import tensorflow as tf
from tensorflow import keras


from tensorflow.keras.preprocessing.text import Tokenizer

sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100)
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index


# Try with words that the tokenizer wasn't fit to
test_data = [
    'i really love my dog',
    'my dog loves my manatee'
]

test_seq = tokenizer.texts_to_sequences(test_data)
print("\nTest Sequence = ", test_seq)
```

```
Test Sequence =  [[4, 2, 1, 3], [1, 3, 1]]
```

Il y a des mots qui ne figurent pas dans le corpus !!!

Exemple

1. Tokenisation

Les mots sont indexées apartir du corpus, ces indices ne changent pas

✓ En cas de besoin :

- Les mots qui ne figurent pas dans le corpus (OOV) alors il aura l'indice =1

Exemple

1. Tokenisation

```
from tensorflow.keras.preprocessing.text import Tokenizer

sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100, oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)
print("\nWord Index = ", word_index)
print("\nSequence = ", sequences)

# Try with words that the tokenizer wasn't fit to
test_data = [
    'i really love my dog',
    'my dog loves my manatee'
]

test_seq = tokenizer.texts_to_sequences(test_data)
print("\nTest Sequence = ", test_seq)
```

OOV out Of Vocabulary

```
Word Index =  {'<OOV>': 1, 'my': 2, 'love': 3, 'dog': 4, 'i': 5, 'you': 6, 'cat': 7, 'do': 8, 'think': 9, 'is': 10, 'amazing': 11}

Sequence =  [[5, 3, 2, 4], [5, 3, 2, 7], [6, 3, 2, 4], [8, 6, 9, 2, 4, 10, 11]]

Test Sequence =  [[5, 1, 3, 2, 4], [2, 4, 1, 2, 1]]
```

Exemple

1. Tokenisation

Les mots sont indexées apartir du corpus, ces indices ne changent pas

✓ En cas de besoin :

- Les mots qui ne figurent pas dans le corpus (OOV) alors il aura l'indice =1
- S'il y aura de padding alors il aura l'indice = 0

Exemple

2. Padding

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100, oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)

padded = pad_sequences(sequences)
print("\nWord Index = " , word_index)
print("\nSequences = " , sequences)
print("\nPadded Sequences:")
print(padded)
```

Exemple

2. Padding

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100, oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)

padded = pad_sequences(sequences)
print("\nWord Index = " , word_index)
print("\nSequences = " , sequences)
print("\nPadded Sequences:")
print(padded)
```

```
Word Index =  {'<OOV>': 1, 'my': 2, 'love': 3, 'dog': 4, 'i': 5, 'you': 6, 'cat': 7, 'do': 8, 'think': 9, 'is': 10, 'amazing': 11}

Sequences =  [[5, 3, 2, 4], [5, 3, 2, 7], [6, 3, 2, 4], [8, 6, 9, 2, 4, 10, 11]]

Padded Sequences:
[[ 0  0  0  5  3  2  4]
 [ 0  0  0  5  3  2  7]
 [ 0  0  0  6  3  2  4]
 [ 8  6  9  2  4 10 11]]
```

Exemple

2. Padding

```
sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100, oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)

padded = pad_sequences(sequences, maxlen=5)
print("\nWord Index = " , word_index)
print("\nSequences = " , sequences)
print("\nPadded Sequences:")
print(padded)


# Try with words that the tokenizer wasn't fit to
test_data = [
    'i really love my dog',
    'my dog loves my manatee'
]

test_seq = tokenizer.texts_to_sequences(test_data)
print("\nTest Sequence = ", test_seq)

padded = pad_sequences(test_seq, maxlen=10)
print("\nPadded Test Sequence: ")
print(padded)
```

On peut preciser la longeur max de padding

Exemple

2. Padding

```
sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100, oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)

padded = pad_sequences(sequences, maxlen=5)
print("\nWord Index = " , word_index)
print("\nSequences = " , sequences)
print("\nPadded Sequences:")
print(padded)


# Try with words that the tokenizer wasn't fit to
test_data = [
    'i really love my dog',
    'my dog loves my manatee'
]

test_seq = tokenizer.texts_to_sequences(test_data)
print("\nTest Sequence = ", test_seq)

padded = pad_sequences(test_seq, maxlen=10)
print("\nPadded Test Sequence: ")
print(padded)
```

On peut preciser la longeur max de padding

```
Word Index =  {'<OOV>': 1, 'my': 2, 'love': 3, 'dog': 4, 'i': 5, 'you': 6, 'cat': 7, 'do': 8, 'think': 9, 'is': 10, 'amazing': 11}

Sequences =  [[5, 3, 2, 4], [5, 3, 2, 7], [6, 3, 2, 4], [8, 6, 9, 2, 4, 10, 11]]

Padded Sequences:
[[ 0  5  3  2  4]
 [ 0  5  3  2  7]
 [ 0  6  3  2  4]
 [ 9  2  4 10 11]]

Test Sequence =  [[5, 1, 3, 2, 4], [2, 4, 1, 2, 1]]

Padded Test Sequence:
[[0 0 0 0 0 5 1 3 2 4]
 [0 0 0 0 0 2 4 1 2 1]]
```

La phrase a été tronquée !!!

Exemple

2. Padding

```
sentences = [
    'I love my dog',
    'I love my cat',
    'You love my dog!',
    'Do you think my dog is amazing?'
]

tokenizer = Tokenizer(num_words = 100, oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)

padded = pad_sequences(sequences, maxlen=5)
print("\nWord Index = " , word_index)
print("\nSequences = " , sequences)
print("\nPadded Sequences:")
print(padded)


# Try with words that the tokenizer wasn't fit to
test_data = [
    'i really love my dog',
    'my dog loves my manatee'
]

test_seq = tokenizer.texts_to_sequences(test_data)
print("\nTest Sequence = ", test_seq)

padded = pad_sequences(test_seq, maxlen=10)
print("\nPadded Test Sequence: ")
print(padded)
```

La solution est le suivante :

```
padded = pad_sequences(sequences, padding='post',
                       truncating='post', maxlen=5)
```

Exemple

3. Cas d'un système de reconnaissance des sentiments

L'objectif de ce cette partie est de faire la classification des textes en sarcastique (ironique) ou non

Nous supposons que les données sont stockées dans des fichier JSON comme suit :


```
{"article_link": "https://politics.theonion.com/boehner-just-wants-wife-to-listen-not-come-up-with-alt-1819574302", "headline": "boehner just wants wife to listen, not come up with alternative debt-reduction ideas", "is_sarcastic": 1}

{"article_link": "https://www.huffingtonpost.com/entry/roseanne-revival-review_us_5ab3a497e4b054d118e04365", "headline": "the 'roseanne' revival catches up to our thorny political mood, for better and worse", "is_sarcastic": 0}

{"article_link": "https://local.theonion.com/mom-starting-to-fear-son-s-web-series-closest-thing-she-1819576697", "headline": "mom starting to fear son's web series closest thing she will have to grandchild", "is_sarcastic": 1}
```

Exemple

3. Cas d'un système de reconnaissance des sentiments

L'objectif de ce cette partie est de faire la classification des textes en sarcastique (ironique) ou non

Nous supposons que les données sont stockées dans des fichier JSON.

Ces données vont être converties en format adéquates pour python (ensemble de listes) :


```
[

{"article_link": "https://politics.theonion.com/boehner-just-wants-wife-to-listen-not-
come-up-with-alt-1819574302", "headline": "boehner just wants wife to listen, not come up
with alternative debt-reduction ideas", "is_sarcastic": 1},

{"article_link": "https://www.huffingtonpost.com/entry/roseanne-revival-
review_us_5ab3a497e4b054d118e04365", "headline": "the 'roseanne' revival catches up to our
thorny political mood, for better and worse", "is_sarcastic": 0},

{"article_link": "https://local.theonion.com/mom-starting-to-fear-son-s-web-series-
closest-thing-she-1819576697", "headline": "mom starting to fear son's web series closest
thing she will have to grandchild", "is_sarcastic": 1}

]
```

Exemple

3. Cas d'un système de reconnaissance des sentiments : premiers traitements

```
import json
```

```
with open("/tmp/sarcasm.json", 'r') as f:
    datastore = json.load(f)

sentences = []
labels = []
urls = []

for item in datastore:
    sentences.append(item['headline'])
    labels.append(item['is_sarcastic'])
    urls.append(item['article_link'])
```

```
import tensorflow as tf

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Exemple

3. Cas d'un système de reconnaissance des sentiments : Tokenisation & padding

```
tokenizer = Tokenizer(oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index

sequences = tokenizer.texts_to_sequences(sentences)
padded = pad_sequences(sequences, padding='post')
print(padded[0])
print(padded.shape)
```

```
[  308 15115   679  3337  2298    48   382  2576 15116     6  2577  8434
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0]



(26709, 40)
```

Exemple

3. Cas d'un système de reconnaissance des sentiments : Trainning & testing

```
training_sentences = sentences[0:training_size]
testing_sentences = sentences[training_size:]
training_labels = labels[0:training_size]
testing_labels = labels[training_size:]
```

Problème :

entrainement après tokenisation

Solution :

Inverser : définir les parties d'entrainement avant et puis faire la tokenisation

Exemple

3. Cas d'un système de reconnaissance des sentiments : Trainning & testing

```
vocab_size = 10000
embedding_dim = 16
max_length = 100
trunc_type='post'
padding_type='post'
oov_tok = "<OOV>"
training_size = 20000
```

```
training_sentences = sentences[0:training_size]
testing_sentences = sentences[training_size:]
training_labels = labels[0:training_size]
testing_labels = labels[training_size:]
```

```
tokenizer = Tokenizer(num_words=vocab_size, oov_token=oov_tok)
tokenizer.fit_on_texts(training_sentences)

word_index = tokenizer.word_index

training_sequences = tokenizer.texts_to_sequences(training_sentences)
training_padded = pad_sequences(training_sequences, maxlen=max_length, padding=padding_type, truncating=trunc_type)

testing_sequences = tokenizer.texts_to_sequences(testing_sentences)
testing_padded = pad_sequences(testing_sequences, maxlen=max_length, padding=padding_type, truncating=trunc_type)
```

Exemple

3. Cas d'un système de reconnaissance des sentiments : Concept de direction

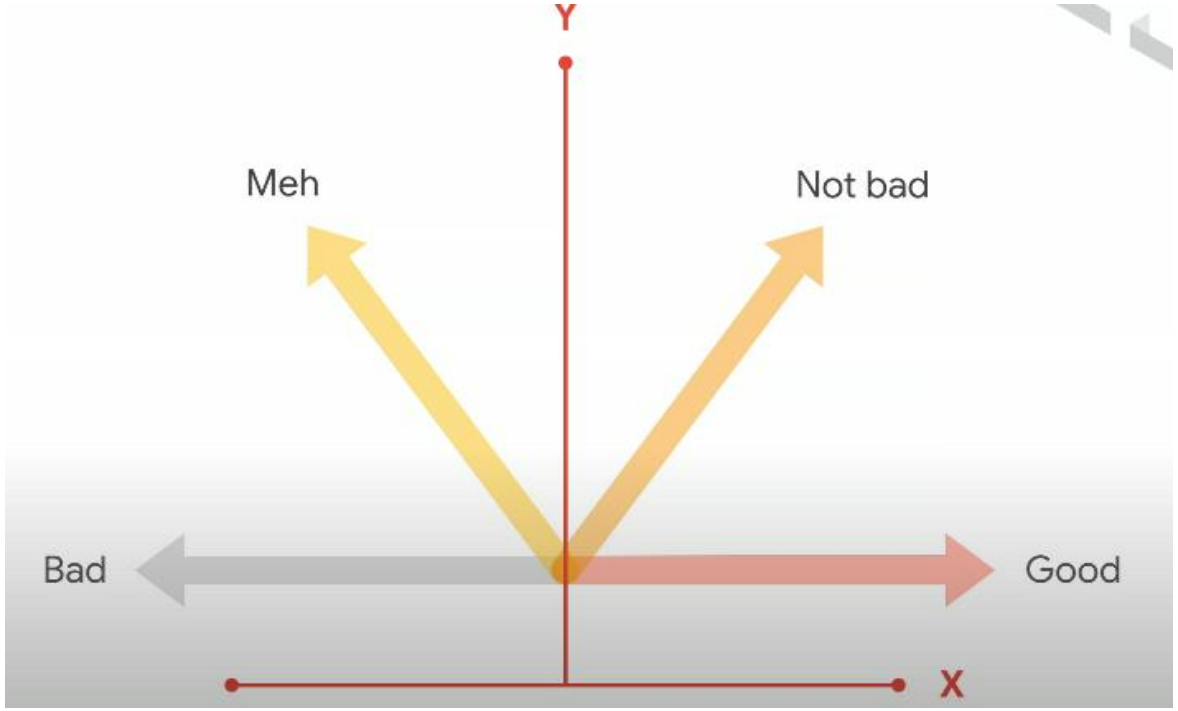
Le sens d'une phrase sera déterminé en se basant sur certains mots clés

Certains mots donnent un sens partagé et favorise la direction de la phrase dans un côté particulier

Il s'agit alors d'analyser la phrase et de construire la flèche qui représente le sens de la phrase dans cet espace qui est à deux dimensions dans notre cas (ironique ou non)

Ceci est fait via un RN

→ Word embedding



Exemple

3. Cas d'un système de reconnaissance des sentiments : Architecture du RN

```
vocab_size = 10000
embedding_dim = 16
max_length = 100
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=max_length),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(24, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
```

- Pour Keras, c'est un Embedding contextuel qui tient compte du corpus défini (local)
- Le mot sera représenté par un vecteur dense de taille 16 (word embedding)
- Les valeurs de ce vecteur seront forgées lors de la phase d'apprentissage via la descente du gradient
- La phrase est un tableau ou chaque élément (mot) est remplacé par son vecteur équivalent (word embedding)

Exemple

3. Cas d'un système de reconnaissance des sentiments : Architecture du RN

```
vocab_size = 10000
embedding_dim = 16
max_length = 100
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=max_length),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(24, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
```

Cette opération est une moyenne sur les valeurs des vecteurs constituants la phrase (words embedding) C'est la moyenne des valeurs sur chaque composante des vecteurs

Toutefois, il faut noter que selon cette conception, le sens des mots est en quelque sorte gardé par le résultat mais l'ordre des mots dans la phrase n'est pas pris en considération

Lien

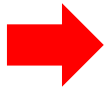
Exemple

3. Cas d'un système de reconnaissance des sentiments : Architecture du RN

```
vocab_size = 10000
embedding_dim = 16
max_length = 100
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=max_length),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(24, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
```

```
model.summary()
```



```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 100, 16)           160000

 global_average_pooling1d (G  (None, 16)               0
 lobalAveragePooling1D)

 dense (Dense)               (None, 24)                408

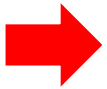
 dense_1 (Dense)             (None, 1)                 25

=================================================================
Total params: 160,433
Trainable params: 160,433
Non-trainable params: 0
```

Exemple

3. Cas d'un système de reconnaissance des sentiments : Entraînement du RN

```
num_epochs = 30
history = model.fit(training_padded, training_labels, epochs=num_epochs, validation_data=(testing_padded, testing_labels), verbose=2)
```

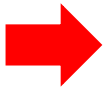


```
Epoch 20/30
625/625 - 3s - loss: 0.0503 - accuracy: 0.9847 - val_loss: 0.7327 - val_accuracy: 0.8240 - 3s/epoch - 5ms/step
Epoch 21/30
625/625 - 3s - loss: 0.0459 - accuracy: 0.9866 - val_loss: 0.8270 - val_accuracy: 0.8208 - 3s/epoch - 5ms/step
Epoch 22/30
625/625 - 3s - loss: 0.0434 - accuracy: 0.9869 - val_loss: 0.8237 - val_accuracy: 0.8208 - 3s/epoch - 4ms/step
Epoch 23/30
625/625 - 3s - loss: 0.0383 - accuracy: 0.9891 - val_loss: 0.9001 - val_accuracy: 0.8170 - 3s/epoch - 5ms/step
Epoch 24/30
625/625 - 3s - loss: 0.0362 - accuracy: 0.9897 - val_loss: 0.8856 - val_accuracy: 0.8211 - 3s/epoch - 5ms/step
Epoch 25/30
625/625 - 3s - loss: 0.0331 - accuracy: 0.9910 - val_loss: 1.0239 - val_accuracy: 0.8140 - 3s/epoch - 5ms/step
Epoch 26/30
625/625 - 3s - loss: 0.0338 - accuracy: 0.9899 - val_loss: 0.9717 - val_accuracy: 0.8156 - 3s/epoch - 5ms/step
Epoch 27/30
625/625 - 3s - loss: 0.0278 - accuracy: 0.9923 - val_loss: 1.0544 - val_accuracy: 0.8155 - 3s/epoch - 5ms/step
Epoch 28/30
625/625 - 3s - loss: 0.0258 - accuracy: 0.9933 - val_loss: 1.0462 - val_accuracy: 0.8150 - 3s/epoch - 5ms/step
Epoch 29/30
625/625 - 3s - loss: 0.0241 - accuracy: 0.9936 - val_loss: 1.1125 - val_accuracy: 0.8152 - 3s/epoch - 5ms/step
Epoch 30/30
625/625 - 3s - loss: 0.0229 - accuracy: 0.9940 - val_loss: 1.1725 - val_accuracy: 0.8126 - 3s/epoch - 5ms/step
```

Exemple

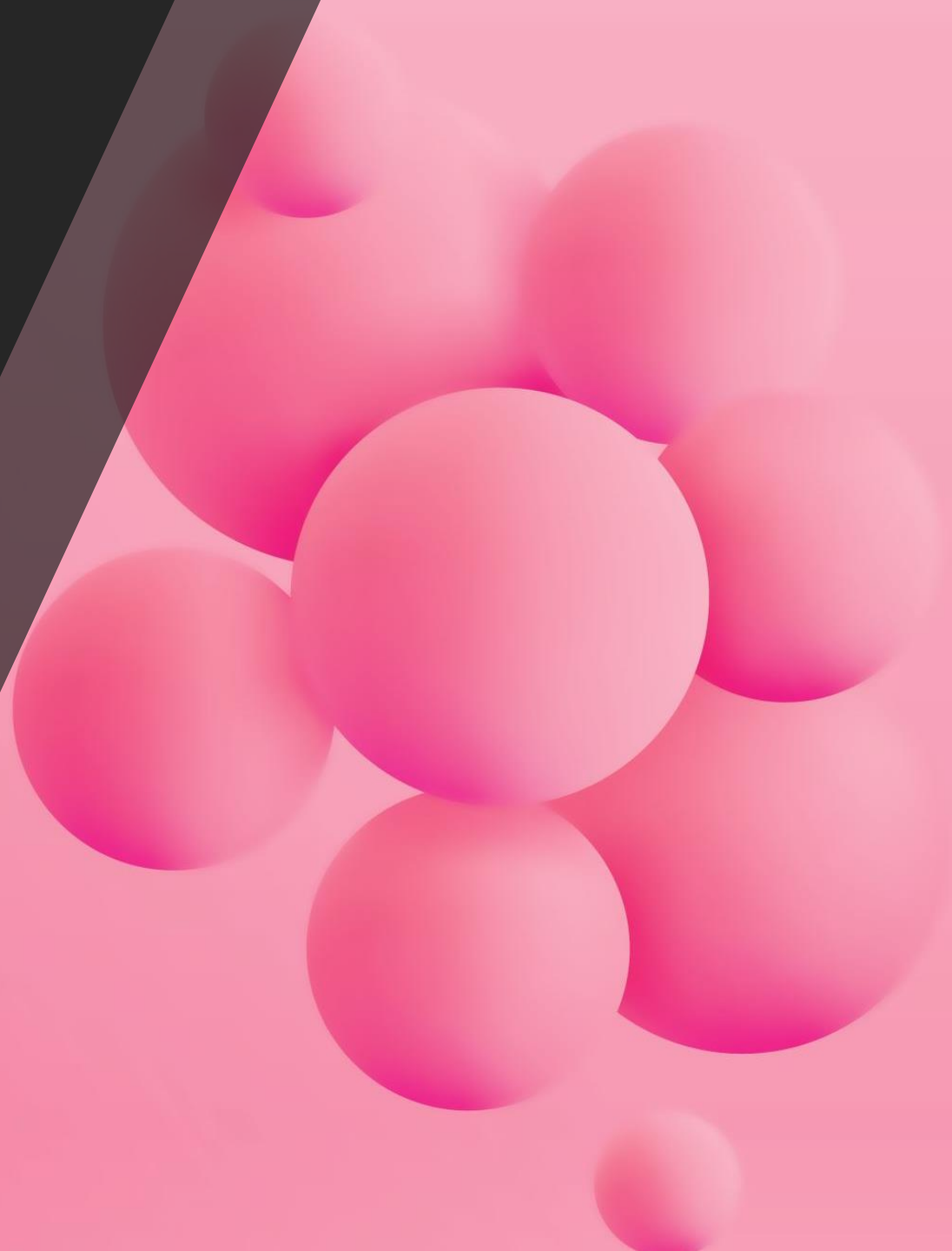
3. Cas d'un système de reconnaissance des sentiments : Validation du RN

```
sentence = ["granny starting to fear spiders in the garden might be real", "game of thrones season finale showing this sunday night"]
sequences = tokenizer.texts_to_sequences(sentence)
padded = pad_sequences(sequences, maxlen=max_length, padding=padding_type, truncating=trunc_type)
print(model.predict(padded))
```



```
[[7.118389e-01]
 [2.020633e-06]]
```

Modèles



Modèles

1. Types

Il y trois types :

a) Sequentiel :

La sortie n'est pas contrainte par une taille fixe (traduction, POS, NER, QA, …)
Ce type se base sur des réseaux de type RNN

b) Attention :

Il permet de mettre en évidence dynamiquement les caractéristiques pertinentes des données d'entrée, qui, en NLP, sont généralement une séquence d'éléments textuels. Comme architecture on peut citer Bahdanau, Luong, BERT, …

c) Transformer :

L'objectif est de gérer facilement les dépendances à longue distance. Calcule des représentations d'entrée et de sortie sans utiliser de RNN ou de CNN et repose entièrement sur self-attention. Ce type se base sur des réseaux de type LSTM

Système de recommandation



Système de recommandation

1. Définition

Un Système de Recommandation (SR) est une forme particulière de filtrage de l'information

Le SR a pour objectif de proposer, à **des utilisateurs (users),** des **éléments (items)** susceptibles de les intéresser

Système de recommandation

2. Applications



Croissance de Netflix, d'Amazon et de YouTube à l'aide du système de recommandation

Système de recommandation

2. Applications

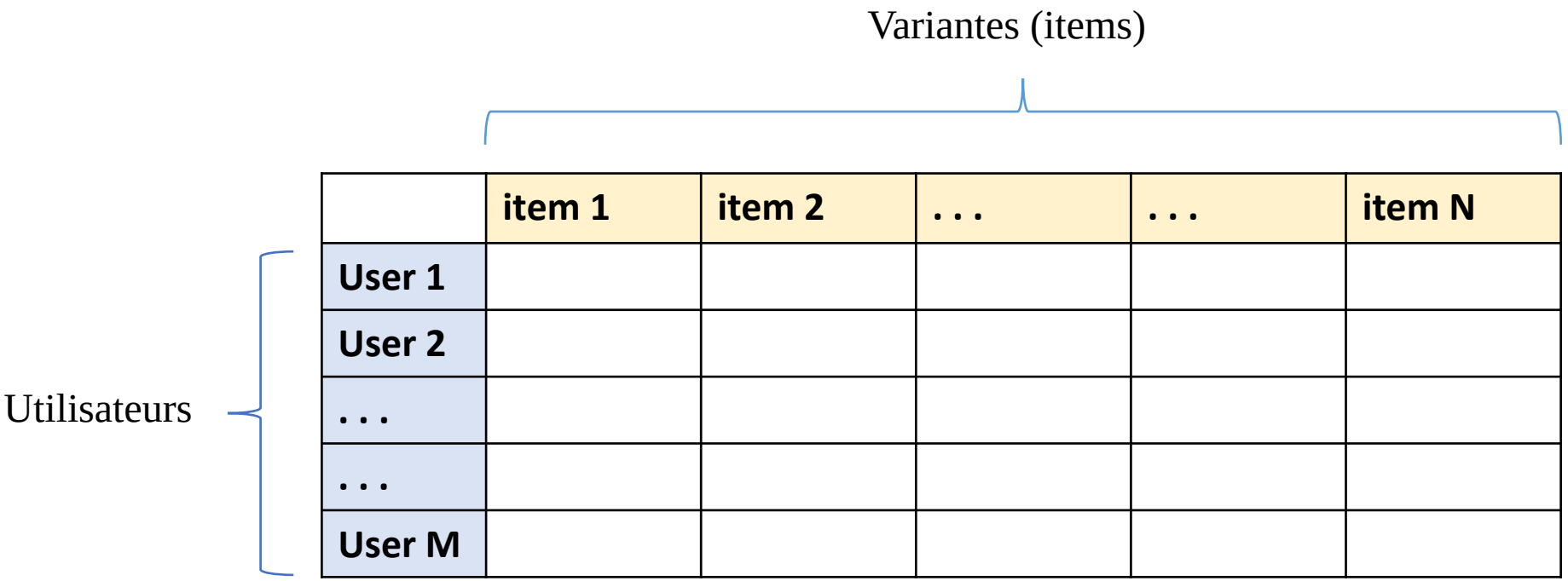
Plusieurs domaines d'application

- Suggestion de contenu :
  - YouTube
  - Netflix
  - Spotify
  - Coursera
  - …
- Suggestion d'amis
- E-commerce
- Election
- Médecine (Prescription des traitements, diagnostic, invention des médicaments, …)
- ….

Système de recommandation

2. Applications

Le système de recommandation est souvent envisageable si on a une sorte de retour d'expérience de la forme suivante :



	item 1	item 2	. . .	. . .	item N
User 1					
User 2					
. . .					
. . .					
User M					

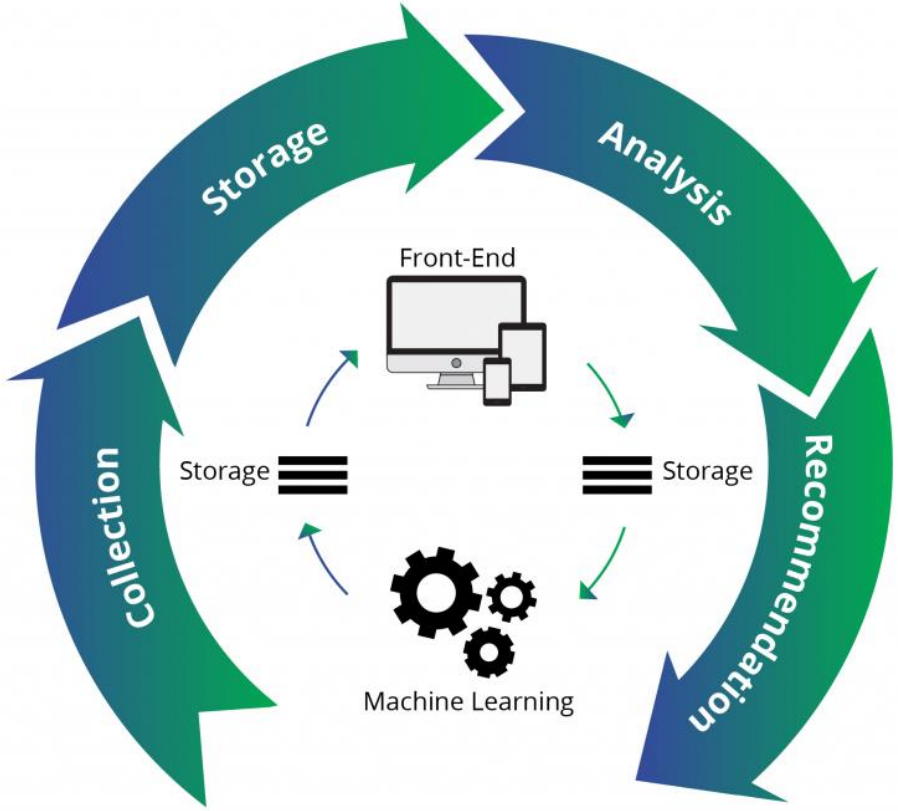
Matrice d'usage

Système de recommandation

3. Motivations

- Objectif premier pour le fournisseur : accroissement des bénéfices, fidéliser les utilisateurs, …
- Intérêt pour l'utilisateur : réduire le temps de recherche, découvrir, …
- Manières de se distinguer
  - Faciliter l'accès aux contenus du système
  - Améliorer la pertinence de ce que l'utilisateur voit
  - Proposer des nouveautés
  - Assurer la diversité

Fonctionnement du SR



Fonctionnement d’un SR

1. Phases

Le SR fonctionne en 3 phases :

- Phase 1 : Collecte des données
- Phase 2 : Analyse et exploitation des données
- Phase 3 : Réalisation des recommandations

Fonctionnement d'un SR

1. Phases

- **Phase 1 : Collecte des données**

❖**Explicite :** les données sont transmises consciemment et volontairement par l'utilisateur.

Exemple : Les notes attribuées aux produits et aux services, les réponses à des enquêtes, etc.

❖**Implicite :** les données sont collectées sans que l'utilisateur n'en soit réellement conscient de ce fait.

Exemple : Les interactions des utilisateurs, des clics, de l'historique de recherche, de la durée passée sur un site, des achats réalisés, etc.

Fonctionnement d’un SR

1. Phases

- **Phase 2 : Analyse et exploitation des données**

Il s’agit d’analyser et de transformer les données de manière à pouvoir les exploiter par la suite. Ces données doivent avoir les représentations adéquates et requises pour alimenter le système de recommandation.

Fonctionnement d’un SR

1. Phases

- **Phase 3 : Réalisation des recommandations**

Il s’agit de fournir des recommandations aux utilisateurs ainsi que de faire des prédictions concernant leurs évaluations à propos de différents items. Le but étant de fournir aux utilisateurs l’expérience la plus intéressante.

Fonctionnement d'un SR

1. Phases

- **Phase 3 : Réalisation des recommandations**

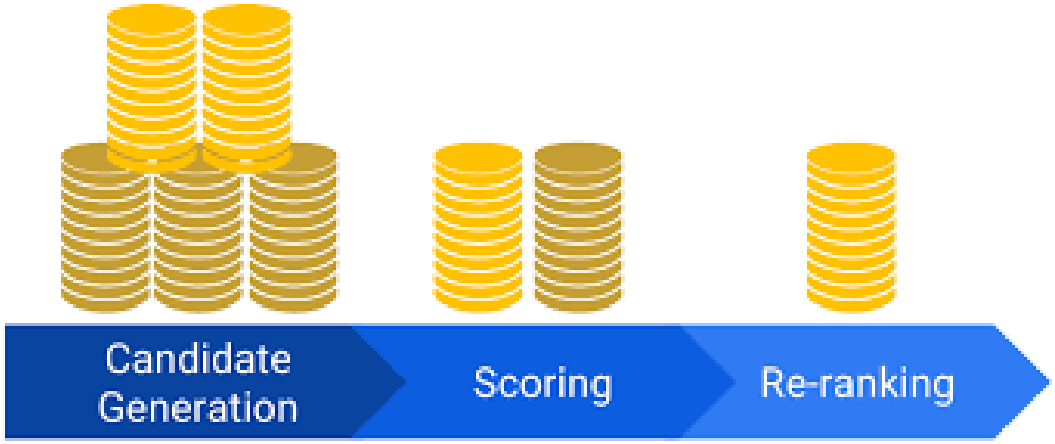
Il s'agit de fournir des recommandations aux utilisateurs ainsi que de faire des prédictions concernant leurs évaluations à propos de différents items. Le but étant de fournir aux utilisateurs l'expérience la plus intéressante.

Cette phase passe par les étapes suivantes :

a) Génération des candidats

b) Notation des candidats (Scoring)

c) Classement des candidats (Ranking)



Fonctionnement d'un SR

2. Recommandation

a) Génération des candidats :

Cette étape se base sur l'une des approches suivantes :

- Les approches type **filtrage collaboratif** *(Collaborative-Filtering)*
- Les approche type **filtrage par le contenu** (*Content-Based*)
- Les approches à base de **factorisation matricielle**
- Les approches utilisant les **DL**
- Les approches **hybrides**, combinant des approches précédentes (*ensemble/hybrid methods*)
- …

Fonctionnement d'un SR

2. Recommandation

b) Notation des candidats :

Une note sera attribuée à chaque candidat. Cette note se base sur le calcul d'une similarité entre des entités (les utilisateurs ou les éléments).

Plusieurs fonctions sont possibles pour le calcul de la similarité :

- Cosine : $\cos(x,y) = x*y/(\|x\|*\|y\|)$
- Produit scalaire $\langle x, y \rangle = \sum_i x_i * y_i$
- Distance euclidienne $\|x-y\| = [\sum_i (x_i * y_i)^2]^{1/2}$
- …

La similarité **cosine** est celle à utiliser et ceci pour des raisons (lien).

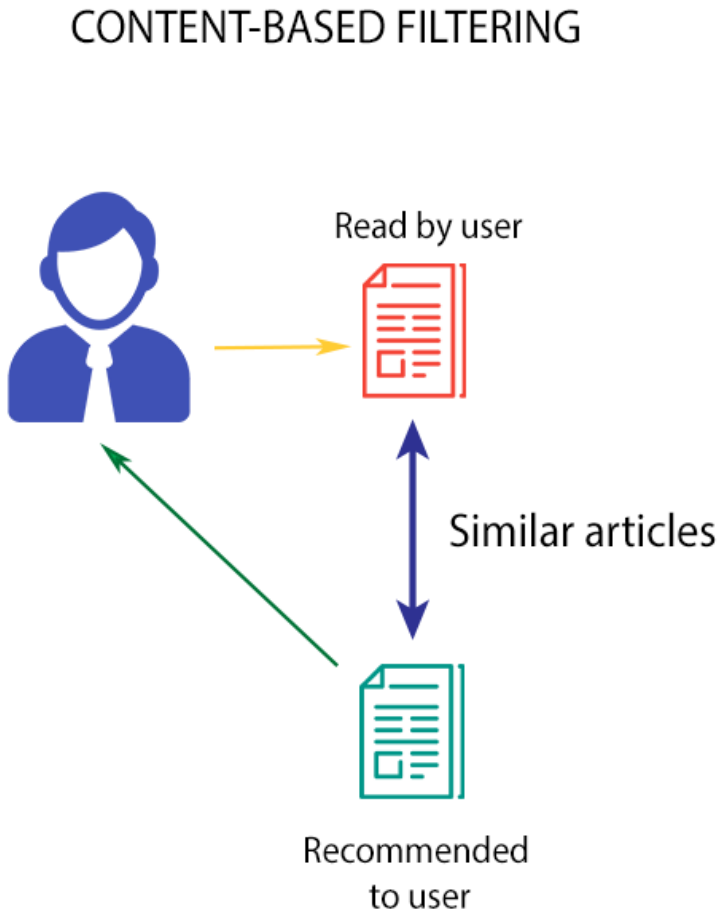
Fonctionnement d’un SR

2. Recommandation

c) Classement des candidats :

Une fois que les notes sont attribuées aux candidats, il faut les trier et choisir celui qui a plus d’affinité pour l’utilisateur en question. Le système sélectionne les premiers éléments et les affiche à l'utilisateur.

Filtrage par le Contenu



Filtrage par le Contenu

1. Principe

Cette approche se base sur la comparaison entre un **élément** et le **profil d'un utilisateur**.

Le but est de proposer des éléments similaires à ceux que l'utilisateur a déjà aimé. Pour cela, il faut qu'on dispose de **caractéristiques descriptives pour les éléments** (exemple : genre de films, artistes d'une chanson, etc.)

Deux sources de données sont à utiliser :

- Les métadonnées (features)
- Un profil utilisateur

Filtrage par le Contenu

2. Etapes

1. Pré-traitement des éléments
   - Représentation vectorielle avec mots-clefs (features)
   - Feature selection
   - …
2. Modélisation des profils
   - Combinaison de feedback et d'attributs
   - Souvent proches des techniques classiques de classement ou régression
   - Pour chaque utilisateur, un modèle est entraîné (”documents étiquetés”)
   - Nearest-neighbors, Classifieur Bayesien naïf, Règles d'associations, ou régression (linéaire, par ex.)
3. Filtrage/recommandation

Filtrage par le Contenu

3. Exemple

Recommandation d'un produit vestimentaire

❖ Représentation vectorielle des éléments

On suppose qu'on a les caractéristiques suivantes : (couleur bleue, couleur rouge, haut, bas, été, hiver)

Donc un pull rouge d'hiver sera représenté par le vecteur V = (0,1,1,0,0,1)

❖ Modélisation du profil

Il faut calculer un **vecteur de profil P** de notre utilisateur cible (client)

Il existe plusieurs méthodes pour le faire, on peut par exemple moyenner l'ensemble des vecteurs des éléments déjà eu

$$P = K * (V_1 + V_2 + V_3 + \dots V_n)$$

K : constante de normalisation

Vj : l'ensemble des éléments (produits) déjà eu (acheté) par l'utilisateur (client)

Filtrage par le Contenu

3. Exemple

Il est maintenant simple de calculer **l'affinité A** d'un utilisateur avec un profil P pour un produit V

A = P *V

Il ne reste qu'à scorer l'ensemble des éléments (produits) non acquis encore par l'utilisateur et à lui proposer ceux qui ont le meilleur score d'affinité. Supposons que l'utilisateur a eu déjà un pull rouge d'hiver et un polo rouge d'été et vu que les caractéristiques sont (couleur bleue, couleur rouge, haut, bas, été, hiver) alors

pull rouge d'hiver	= (0,	1,	1,	0,	0,	1)	
polo rouge d'été	= (0,	1,	1,	0,	1,	0)	d'où

P = K*[(0, 1, 1, 0, 0, 1) + (0, 1, 1, 0, 1, 0)]

= K*(0, 2, 2, 0, 1, 1)

Quelle serait alors l'affinité de l'utilisateur avec un pantalon rouge d'été et une écharpe rouge d'hiver ?

Filtrage par le Contenu

3. Exemple

Il est maintenant simple de calculer **l'affinité A** d'un utilisateur avec un profil P pour un produit V

A = P *V

Il ne reste qu'à scorer l'ensemble des éléments (produits) non acquis encore par l'utilisateur et à lui proposer ceux qui ont le meilleur score d'affinité. Supposons que l'utilisateur a eu déjà un pull rouge d'hiver et un polo rouge d'été et vu que les caractéristiques sont (couleur bleue, couleur rouge, haut, bas, été, hiver) alors

pull rouge d'hiver	= (0,	1,	1,	0,	0,	1)	
polo rouge d'été	= (0,	1,	1,	0,	1,	0)	d'où

P = K*[(0, 1, 1, 0, 0, 1) + (0, 1, 1, 0, 1, 0)]

= K*(0, 2, 2, 0, 1, 1)

Quelle serait alors l'affinité de l'utilisateur avec un pantalon rouge d'été et une écharpe rouge d'hiver ?

$A_{pantalon}$ = P*V = K*(0, 2, 2, 0, 1, 1) (0, 1, 0, 1, 1, 0) = K * (0+2+0+0+1+0) = 3*K

$A_{écharpe}$ = P*V = K*(0, 2, 2, 0, 1, 1) (0, 1, 1, 0, 0, 1) = K * (0+2+2+0+0+1) = 5*K ⇒ recommander l'écharpe

Filtrage par le Contenu

4. Evaluation

☑ Avantages

✓Le modèle n'a pas besoin de données sur les autres utilisateurs. Cela facilite l'adaptation à un grand nombre d'utilisateurs (easy user scaling), c'est souvent le cas réel, le nombre d'items est très grand par rapport au nombre d'utilisateurs

✓Le modèle peut capturer les intérêts spécifiques d'un utilisateur et peut lui recommander des éléments convenablement.

✗ Inconvénients

❖Aucune donnée n'est nécessaire à propos des autres utilisateurs, ceci dit que la similarité entre les expériences des utilisateurs n'est pas exploitée.

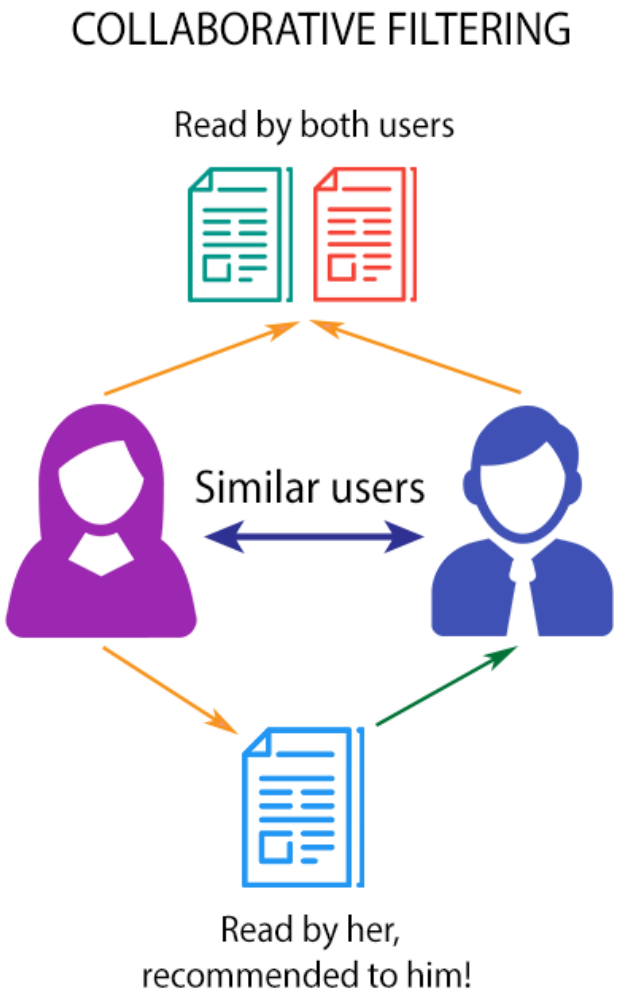
❖La conception des métadonnées (features) dépend des experts du domaine en question. Chose qu'on peut la surmonter par le DL mais quelles métadonnées sont à utiliser !!!

❖Effet de bulle : recommandation des choses similaires

❖Faible diversité

Filtrage Collaboratif

Similar users favor similar items



Filtrage Collaboratif

1. Principe

Comparer **l'ensemble des utilisateurs** d'une application ou d'un site pour avoir une approche collaborative.

Hypothèse :

Si deux utilisateurs partagent le même goût sur un élément (produit par exemple)

Alors ils ont PLUS de chance de partager la même opinion sur un autre élément QUE deux utilisateurs pris au hasard.

Mise en place :

Pour la mise en place de ce type de filtrage, on peut :

- soit calculer des similarités entre éléments (Item-Based)
- soit calculer des similarités entre utilisateurs (User-Based)
- Soit se baser sur les éléments et les utilisateurs (Embedded)

Filtrage Collaboratif

2. Item-Based

Principe : les éléments qui se ressemblent sont utilisés par des utilisateurs similaires

- On calcule la similarité entre toutes paires d'items
- On estime l'affinité d'un utilisateur pour un item en analysant les autres items et on lui propose l'élément le mieux scoré parmi ceux qu'il n'a pas

- La fonction de similarité adoptée est la similarité cosine:

$$sim(x,y) = \cos(x,y)$$

$$= x.y/(\|x\|*\|y\|)$$

Filtrage Collaboratif

2. Item-Based

Exemple : Matrice d’achat (1=achat, 0=non-achat)

Que proposer à Céline ? Chaussures ou Veste ???

	Robe	Chaussures	Veste
Alice	1	1	0
Bob	0	1	1
Céline	1	0	0

Filtrage Collaboratif

2. Item-Based

Exemple : Matrice d'achat (1=achat, 0=non-achat)

	Robe	Chaussures	Veste
Alice	1	1	0
Bob	0	1	1
Céline	1	0	0

L'objectif est donc de calculer la **similarité entre les différents éléments** (produits)

sim(robe, chaussures) = (1*1+0*1+1*0) / (√(2)*√(2)) = 1/2 = 0.5

sim(robe, veste) = (1*0+0*1+1*0) / (√(2)*√(1)) = 0

sim(veste, chaussures) = (0*1+1*1+0*0) / (√(1)*√(2)) = 1/ √ 2 = 0.7

Filtrage Collaboratif

2. Item-Based

Exemple : Matrice d'achat (1=achat, 0=non-achat)

sim(robe, chaussures) = 0.5

sim(robe, veste) = 0

sim(veste, chaussures) = 0.7

	Robe	Chaussures	Veste
Alice	1	1	0
Bob	0	1	1
Céline	1	0	0

Donc on peut maintenant estimer un score S_{ij} **d'affinité** entre l'utilisateur i et l'élément j

$S_{ij} = K * [sim(P_j,P_1)*V_{i1} + sim(P_j,P_2)*V_{i2} + \ldots + sim(P_j,P_n)*V_{in}]$

K : constante de normalisation

V_{ij} : valeur du couple utilisateur i / élément j

P_j : vecteur de l'élément j (colonne j de la matrice)

Ensuite on peut proposer à un utilisateur l'élément le mieux scoré pour lui parmi ceux qu'il n'a pas

Filtrage Collaboratif

2. Item-Based

Exemple : Matrice d'achat (1=achat, 0=non-achat)

sim(robe, chaussures) = 0.5

sim(robe, veste) = 0

sim(veste, chaussures) = 0.7

	Robe	Chaussures	Veste
Alice	1	1	0
Bob	0	1	1
Céline	1	0	0

Vaut-il mieux proposer une veste ou des chaussures à Céline ?

$S_{céline,\ veste} = sim(P_{veste},P_{robe})*V_{céline,\ robe} + sim(P_{veste},P_{chaussures})*V_{céline,\ chaussures}$

$= 0 * 1$

$= 0$

$S_{céline,\ chaussures} = sim(P_{chaussures},P_{robe})*V_{céline,\ robe} + sim(P_{chaussures},P_{veste})*V_{céline,\ veste}$

$= 0{,}5 * 1$

$= 0{,}5$

⇒ il est donc préférable de recommander des chaussures à Céline



Filtrage Collaboratif

3. User-Based

Principe : les utilisateurs qui se ressemblent ont tendance d'avoir des éléments similaires

- On calcule la similarité entre toutes paires d'utilisateurs
- On estime l'affinité d'un utilisateur envers un item en regardant les relations à cet item avec les autres utilisateurs et on lui propose l'élément le mieux scoré parmi ceux qu'il n'a pas

Filtrage Collaboratif

3. User-Based

Exemple : Matrice d'achat (1=achat, 0=non-achat)

	Robe	Chaussures	Veste
Alice	1	1	0
Bob	0	1	1
Céline	1	0	0

L'objectif est donc de calculer la **similarité entre les différents utilisateurs**

sim(Alice, Céline) = (1*1+1*0+0*0) / (√(2)*√(1)) = 1/ √ 2 = 0.7

sim(Bob, Céline) = (0*1+1*0+1*0) / (√(2)*√(1)) = 0/ √ 2 = 0

Filtrage Collaboratif

3. User-Based

Exemple : Matrice d'achat (1=achat, 0=non-achat)

	Robe	Chaussures	Veste
Alice	1	1	0
Bob	0	1	1
Céline	1	0	0

sim(Alice, Céline) = 0.7
sim(Bob, Céline) = 0

Donc on peut maintenant estimer un score S_{ij} **d'affinité** entre l'utilisateur i et l'élément j

$S_{ij} = K * [sim(C_i,C_1)*V_{1j} + sim(C_i,C_2)*V_{2j} + \ldots + sim(C_i,C_n)*V_{nj}]$

K : constante de normalisation

V_{ij} : valeur du couple utilisateur i / élément j

C_i : vecteur de l'utilisateur i (ligne i de la matrice)

Ensuite on peut proposer à un utilisateur l'élément le mieux scoré pour lui parmi ceux qu'il n'a pas

Filtrage Collaboratif

3. User-Based

Exemple : Matrice d'achat (1=achat, 0=non-achat)

	Robe	Chaussures	Veste
Alice	1	1	0
Bob	0	1	1
Céline	1	0	0

sim(Alice, Céline) = 0.7
sim(Bob, Céline) = 0

Vaut-il mieux proposer une veste ou des chaussures à Céline ?

$S_{céline,\ veste} = sim(C_{céline},C_{alice})*V_{alice,\ veste} + sim(C_{céline},C_{bob})*V_{bob,\ veste}$

$= 0,7 * 0 + 0 * 1$

$= 0$

$S_{céline,\ chaussures} = sim(C_{céline},C_{alice})*V_{alice,\ chaussures} + sim(C_{céline},C_{bob})*V_{bob,\ chaussures}$

$= 0,7 * 1 + 0 * 1$

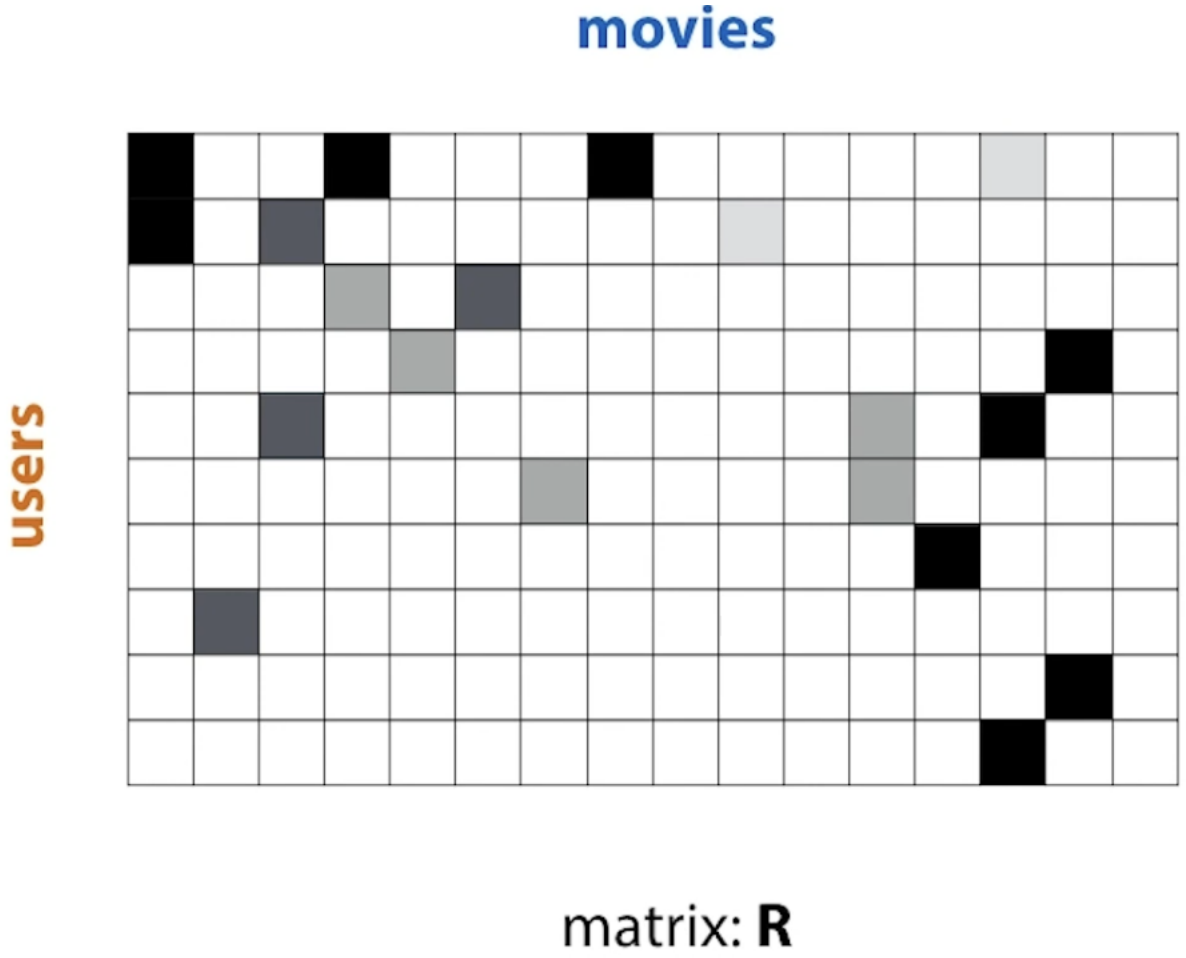
$= 0,7$

⇒ il est donc préférable de recommander des chaussures à Céline

Filtrage Collaboratif

4. Embedded

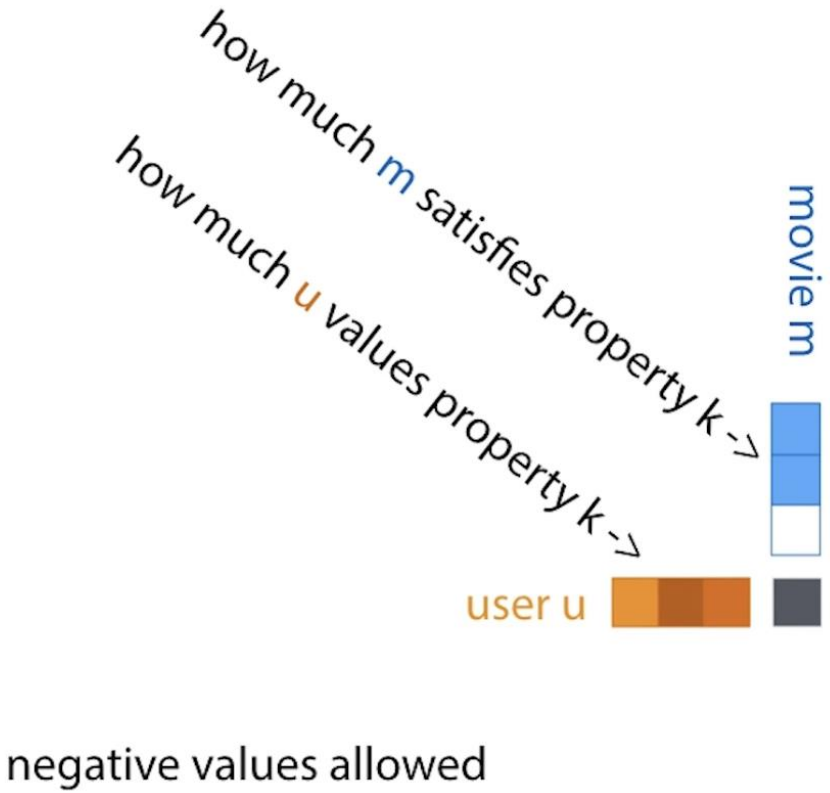
La matrice est creuse



Filtrage Collaboratif

4. Embedded

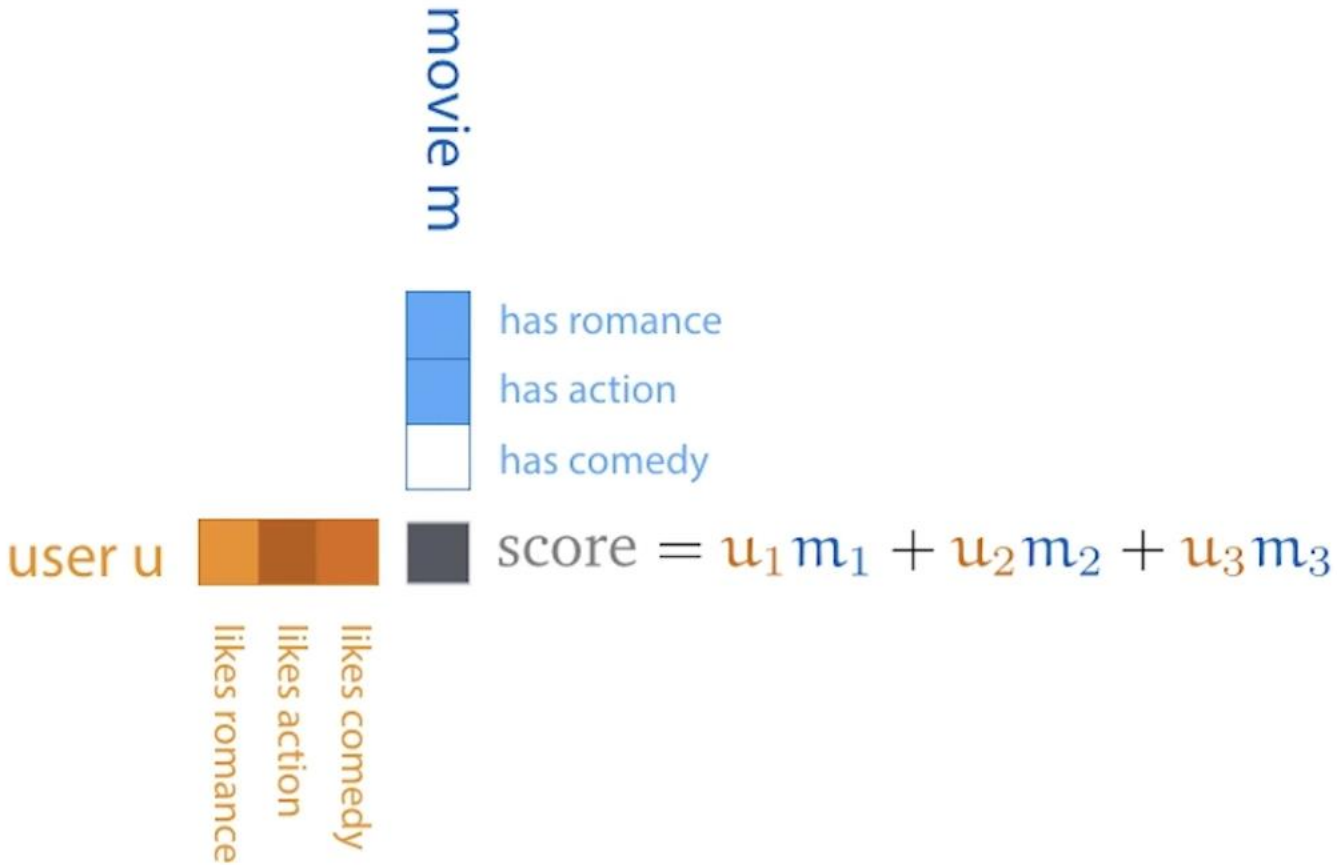
L’utilisateur sera représenté par un vecteur encastré et aussi chaque élément



Filtrage Collaboratif

4. Embedded

Un score est calculé comme suit :



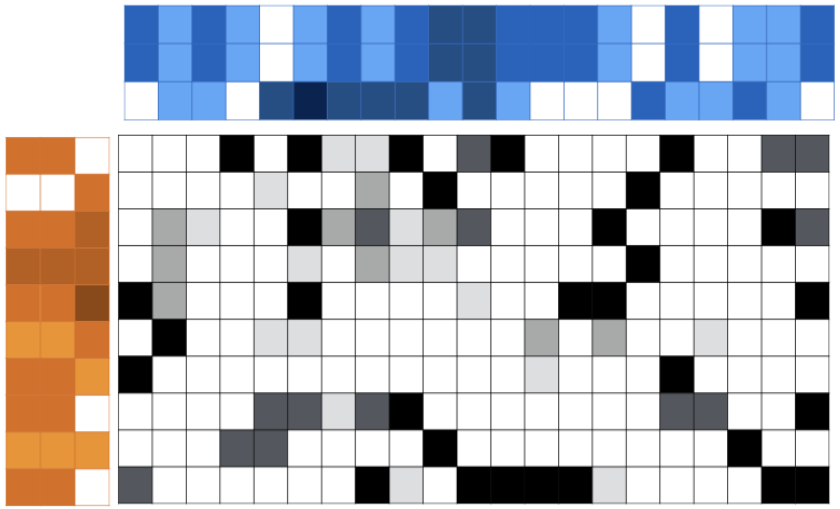
Filtrage Collaboratif

4. Embedded

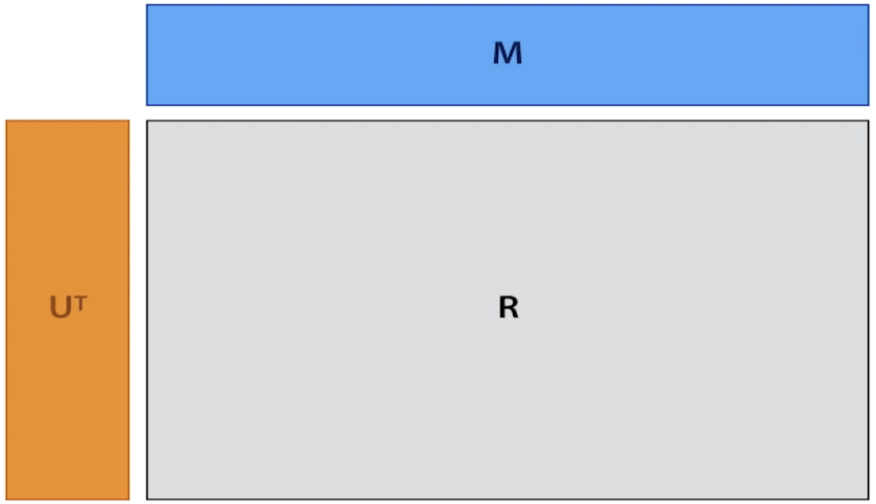
La matrice n'est pas dense et l'objectif est de prédire les valeurs manquantes de la matrice R (Ranking)

Il est alors nécessaire d'optimiser les valeurs de vecteur U et de vecteur M de telle manière à approximer au mieux les valeurs de R. Ceci peut être fait via un modèle d'apprentissage.

N.B. Les valeurs manquantes peuvent être remplacées par des valeurs à définir selon le contexte.



Choose U and M so that $U^T M \approx R$



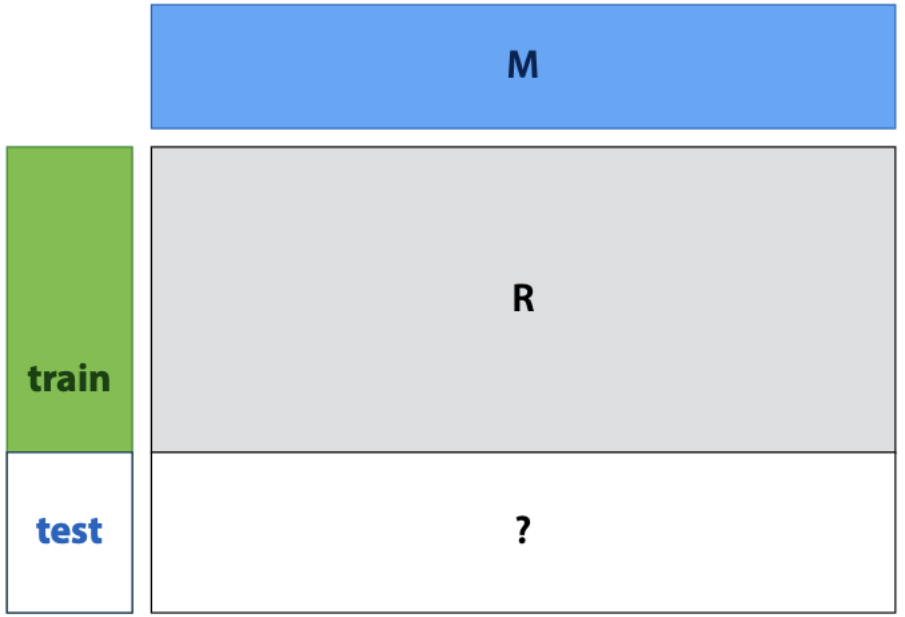
Filtrage Collaboratif

4. Embedded

Il est important d'examiner attentivement le protocole de validation du SR. En d'autres termes : comment retenir les données de test pour s'entraîner ?

Protocole 1 : ne retenir que certains utilisateurs

Cela ne fonctionne pas. En effet, si les utilisateurs ne figurent pas durant le training, alors le SR n'apprend pas leurs représentations encastrée, ce qui signifie qu'il est impossible de générer de prédictions.



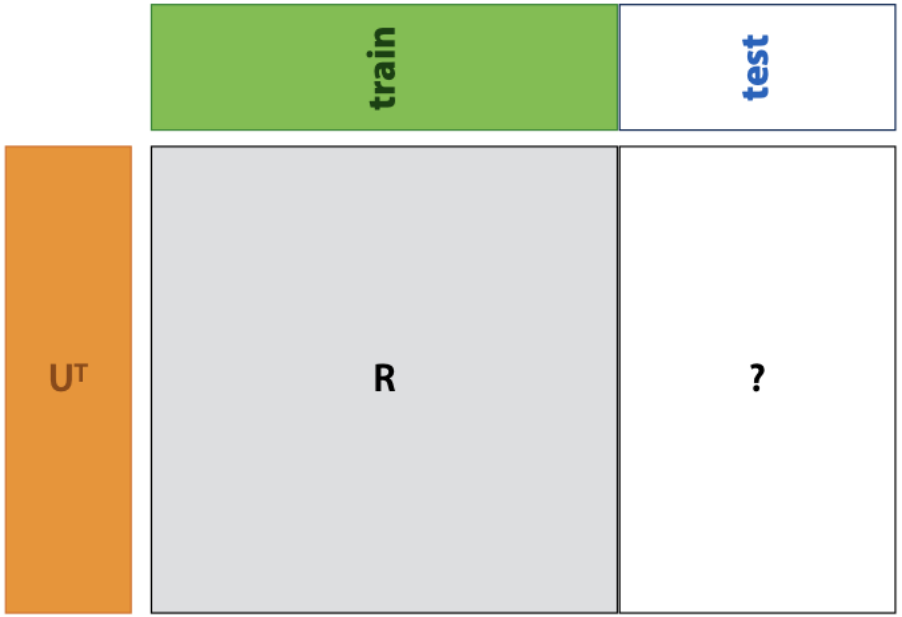
Filtrage Collaboratif

4. Embedded

Il est important d'examiner attentivement le protocole de validation du SR. En d'autres termes : comment retenir les données de test pour s'entraîner ?

Protocole 2 : ne retenir que certains films

On aura le même problème et donc il est impossible de générer des prédictions.



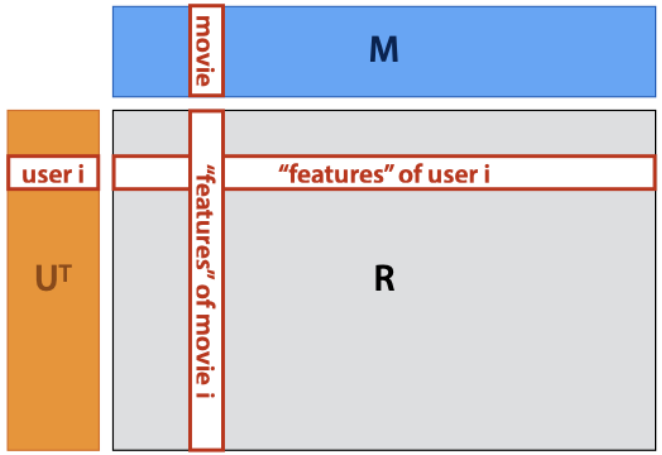
Filtrage Collaboratif

4. Embedded

Il est important d'examiner attentivement le protocole de validation du SR. En d'autres termes : comment retenir les données de test pour s'entraîner ?

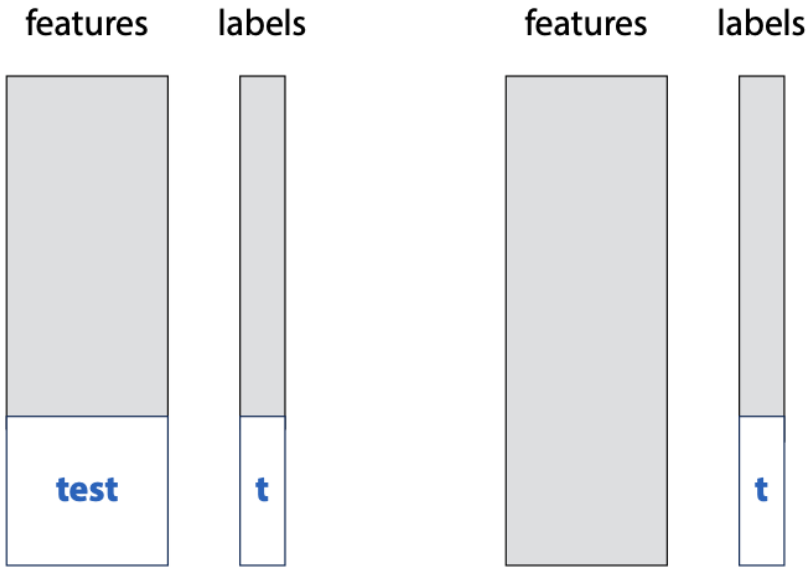
Solution

Mixed features + Apprentissage transductif



the features of our users are their ratings over all movies

the features of our movies are their ratings from all users.



Filtrage Collaboratif

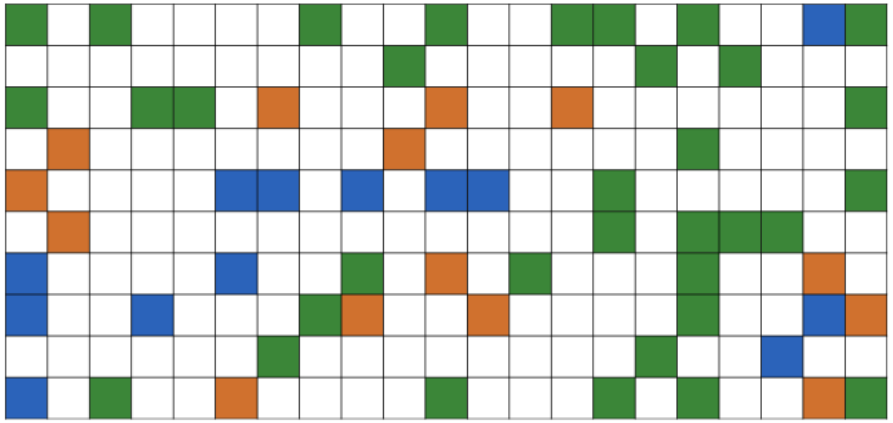
4. Embedded

Il est important d'examiner attentivement le protocole de validation du SR. En d'autres termes : comment retenir les données de test pour s'entraîner ?

Solution

Mixed features + Apprentissage transductif

recommendation: train, validation, test



Filtrage Collaboratif

5. Evaluation

☑ Avantages

✓Facilité d'implémentation et d'interprétabilité

✓Recommandation des produits diversifié à un utilisateur

✕ Inconvénients

❖Phase offline couteuse en temps et espace

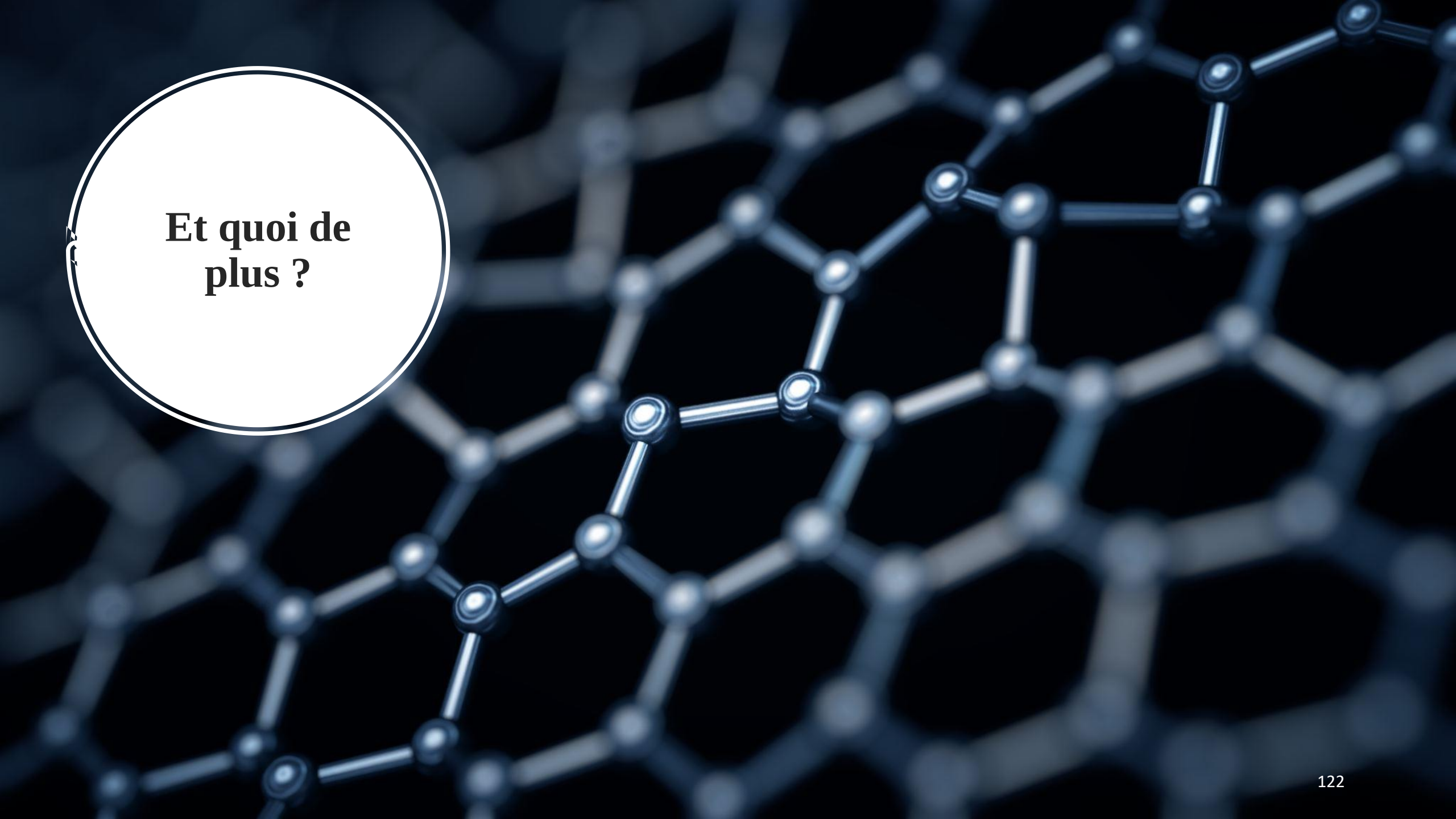
❖Trop peu d'utilisateurs ou d'items similaires : Couverture limitée (matrice creuse)

❖Démarrage à froid (cold start) : si un produit est nouveau et que personne ne l'a encore eu, alors on n'a aucune chance de le recommander

Autres approches

il existe d’autres types d’approches :

- Approximation par représentation latente
- Décomposition en valeurs singulières SVD


Et quoi de plus ?

& +

1. Labs

- Reconnaissance des sentiments

https://colab.research.google.com/github/lmoroney/dlaicourse/blob/master/TensorFlow%20In%20Practice/Course%203%20-%20NLP/Course%203%20-%20Week%202%20-%20Lesson%202.ipynb#scrollTo=cG8-ArY-qDcz

- Créer la poésie

https://colab.research.google.com/github/lmoroney/dlaicourse/blob/master/TensorFlow%20In%20Practice/Course%203%20-%20NLP/Course%203%20-%20Week%204%20-%20Lesson%202%20-%20Notebook.ipynb#scrollTo=6Vc6PHgxa6Hm

- Bow & pipeline

https://colab.research.google.com/drive/1n8jcgGWfmjrTJsl8vKXqzROgU-mRVA32#scrollTo=E_y4-5MbvA1j

2. Bibliothèques

1. Natural Language Toolkit (NLTK)

2. TextBlob

3. CoreNLP

4. Gensim

5. spaCy

6. polyglot

7. scikit–learn

8. Pattern

2. Bibliothèques

NLP

Les ressources suivantes sont quelques-unes des boîtes à outils open source les plus populaires pour le traitement du langage naturel :

- **Stanford CoreNLP :** une boîte à outils basée sur Java d'outils d'analyse linguistique pour le traitement de texte en langage naturel. CoreNLP a été développé par l'Université de Stanford.
- **NLTK** : Le Natural Language Toolkit, ou NLTK en abrégé, est une suite open-source de bibliothèques pour le traitement symbolique et statistique du langage naturel pour l'anglais. Il a été développé par l'Université de Pennsylvanie.
- **Gensim** : une boîte à outils open source basée sur Python qui se concentre sur l'espace vectoriel et la modélisation thématique de documents texte.
- **spaCy** : une boîte à outils hautes performances basée sur Python pour le traitement avancé du langage naturel. SpaCy est open source et pris en charge par Explosion AI.
- **OpenNLP** : une boîte à outils d'apprentissage automatique open source pour le traitement de texte en langage naturel. OpenNLP est sponsorisé par le projet Apache.
- **AllenNLP** : une bibliothèque de recherche NLP intégrée à PyTorch.

2. Bibliothèques

Reconnaissance de la parole

- **Sphinx** : une boîte à outils ASR développée par l'Université Carnegie Mellon, axée sur
- production et développement d'applications.

- **Kaldi** : un framework ASR C++ open-source, conçu pour le traitement de la parole axé sur la recherche ainsi que pour un usage professionnel.

- **ESPnet** : un cadre ASR de bout en bout basé sur l'apprentissage en profondeur, inspiré de Kaldi et écrit avec les backends PyTorch et Chainer.

2. Bibliothèques

Traitement audio

- •**SoX** : une boîte à outils et une bibliothèque de manipulation audio. Il implémente de nombreux formats de fichiers et est utile pour lire, convertir et manipuler des fichiers audio [NB18].

- **LibROSA** : un package Python pour l'analyse audio, couramment utilisé pour les fonctionnalités extraction et traitement des signaux numériques (DSP) [McF+15].

2. Bibliothèques

Outils et bibliothèques supplémentaires

- **KenLM** : une boîte à outils de modélisation de langage n-gram hautes performances généralement intégrée aux frameworks ASR.

- •**LIME** : explication indépendante du modèle interprétable local (LIME), un explicateur local et indépendant du modèle pour les modèles d'apprentissage automatique et d'apprentissage en profondeur.